
Provable Benefits of RLVR over SFT for Reasoning Models: Learning to Backtrack Efficiently

Stanley Wei¹ Juno Kim²

Abstract

Recent advances in large language models (LLMs) have demonstrated that reinforcement fine-tuning of pretrained base models can lead to significant gains in reasoning performance at inference time. In this work, we theoretically analyze why reinforcement fine-tuning induces better reasoning ability than purely supervised fine-tuning (SFT) methods. We model chain-of-thought (CoT) reasoning as a pathfinding problem on graphs and compare the popular method of reinforcement learning with verifiable rewards (RLVR) against traditional SFT. We prove that SFT, when trained on golden shortest paths without negative examples, fails to learn how to efficiently backtrack. In contrast, an RLVR-trained model can learn how to efficiently backtrack from dead ends using only outcome reward. This leads to an exponential separation in inference-time compute between the two methods, and demonstrates that RLVR leads the model to learn the location of difficult decisions in a reasoning chain, ultimately allowing for better allocation of inference-time compute. Finally, we show that the reasoning traces of an RLVR model can be distilled to train a base model to backtrack efficiently as well.

1. Introduction

Modern large language models (LLMs) are trained to achieve strong reasoning capabilities on a wide range of tasks such as mathematical problem-solving and code generation. In the context of LLMs, *reasoning* refers to the generation of long chain-of-thought (CoT) which mimics the step-by-step nature of human reasoning to solve complex logical problems (Wei et al., 2022; Lightman et al.,

2023). Reasoning models treat such tasks as a sequential multi-step decision process and deploy strategies utilizing additional test-time compute budget, such as sampling, tree search, aggregation, and backtracking. This approach has proved to yield efficient and scalable gains over initial pre-training (Snell et al., 2024; Muennighoff et al., 2025), and has been adopted with great success in various frontier and open-source models (OpenAI, 2024; Shao et al., 2024; Guo et al., 2025).

In order to learn or strengthen these desired inference-time reasoning behaviors, LLMs must undergo stages of post-training (Kumar et al., 2025; Xu et al., 2025). The post-training procedure typically follows one of two main strategies: supervised fine-tuning (SFT) on expert demonstrations, and reinforcement learning (RL) on feedback from a reward model or task verifier. SFT uses an off-policy dataset of demonstrations created or annotated by an expert (typically a human or a more powerful model) and trains the base model to imitate these responses. While SFT has been the de facto method for post-training, it is prone to memorization or overfitting (Chu et al., 2025a) and can induce uninformative pseudo-reasoning paths (Chen et al., 2025). Moreover, curating high-quality expert demonstrations can be expensive and time-consuming, depending on the nature of the task.

In contrast, methods such as reinforcement learning with verifiable rewards (RLVR) (Wen et al., 2025; DeepSeek-AI et al., 2025a) and reinforcement learning from human feedback (RLHF) (Christiano et al., 2023; Ouyang et al., 2022) learn from reward models or verifiers via on-policy exploration. The RL approach has been argued to generalize more effectively and have the potential to unlock entirely new reasoning capabilities (Chu et al., 2025a; Wang et al., 2025; Zhu et al., 2025). Nevertheless, we still lack a principled understanding of the differences between the two methods, or a theoretical framework under which to compare various post-training algorithms.

In this paper, we focus on *backtracking ability* as a way to distinguish the effectiveness of post-training methods. Backtracking is a key element of human problem-solving: when a line of approach is revealed to be incorrect or unhelpful, one returns to an earlier decision point and explores an

¹Princeton University ²University of California, Berkeley. Correspondence to: Stanley Wei <stanley.wei@princeton.edu>.

alternative branch. This greatly reduces complexity of the effective search space. Empirically, backtracking has been shown to be greatly beneficial to LLM reasoning ability, either implicitly in the chain of thought (Cai et al., 2025), or explicitly with backtrack tokens (Yang et al., 2025b) or rolling back sequence generation (Singh et al., 2025). Hence we are motivated to ask:

Which post-training method teaches the base model to efficiently backtrack at inference time?

Our contributions. We approach this question by modeling CoT reasoning as pathfinding on graphs, a sandbox commonly studied in the literature (Sanford et al., 2024; Bachmann & Nagarajan, 2025; Kim et al., 2025). While existing works have studied the benefits of backtracking from an information or sampling perspective (Shalev-Shwartz & Shashua, 2025; Rohatgi et al., 2025), we provide a novel dynamical characterization of running SFT versus RLVR. We design a multigraph similar to the path-star graph (Bachmann & Nagarajan, 2025) with W branches of depth K , which the pretrained world model – a linear-softmax bigram over edges, or a trigram over nodes – must learn to navigate by backtracking from failed branches. Our results are summarized as follows.

- We prove that SFT trained only on golden shortest paths does not learn any backtracking strategy, while the RLVR-trained model learns to backtrack consistently using only outcome reward with a length penalty in finite time.
- We show an exponential test-time compute separation: after convergence, the RLVR model can reach any target in $\Theta(WK)$ expected time, while the SFT model requires $\Theta(WL^K)$ time.
- We further show that distilling RLVR-generated reasoning traces to a base model via supervised learning transfers efficient backtracking, recovering $\Theta(WK)$ inference-time compute.

Intuitively, SFT on expert solutions trains the base model to continue along a gold path with no exposure to dead ends. Hence, SFT need not learn an efficient retreat strategy. In contrast, on-policy RLVR necessarily generates and trains on the model’s own unsuccessful partial rollouts. This exploration produces gradient signal not only about good forward actions, but also backtracking actions when the model has committed to a poor branch. We formalize this difference via a dynamical analysis of gradient flow (for SFT) and sign policy-gradient flow (for RLVR). Our results provide theoretical support for the importance of backtracking data for reasoning in both RL and distillation.

All proofs are deferred to the appendix.

2. Related Work

Backtracking in LLM reasoning. Backtracking has empirically been used to quantify and improve reliability and efficiency in LLM reasoning. Inference-time search frameworks such as Tree-of-Thoughts explicitly explore and prune a branching space of intermediate thoughts, leading to depth-first or best-first search with backtracking (Yao et al., 2023; Long, 2023), generalized by Graph-of-Thoughts methods (Besta et al., 2024). Qin et al. (2025) study when sequential search with backtracking benefits over parallel sampling. In particular, they empirically show that models with backtracking capabilities benefit greatly from RL finetuning on reasoning tasks, which agrees with our theoretical results. Moreover, Singh et al. (2025) propose preemptive backtracking guided by in-context value verification to identify and focus resampling from suspected failure points. Yang et al. (2025b) study encouraging models to decide when to revert during reasoning via introducing an explicit backtrack token, aiming to internalize structured backtracking.

From a theoretical perspective, Shalev-Shwartz & Shashua (2025) show the necessity of search and backtracking for certain graph search tasks and parity problems, and describe a learning method which builds a search tree with explicit backtracking. Rohatgi et al. (2025) propose a test-time sampling and backtracking algorithm which uses process rewards, which mixes in time quadratic in the depth of the search tree, allowing for exact sampling from a target distribution on the leaves. However, these works do not study how to learn such behavior via post-training, which is the focus of our dynamical analysis. Kim et al. (2025) propose a cluster graph model of reasoning and analyze CoT pathfinding as a metastable Markov process. They also provide convergence guarantees for a simple RL method (proximal policy optimization); however, they do not study RLVR or SFT, and do not consider backtracking in CoT.

RL versus SFT. Recent empirical studies have highlighted significant qualitative differences between models post-trained via SFT and those with RL. Chu et al. (2025b) provide a comparative analysis on an arithmetic-based reasoning task that suggests SFT tends to memorize the distribution of training demonstrations, whereas RL facilitates better generalization to out-of-distribution (OOD) tasks. This distinction is further supported by Shenfeld et al. (2025), in which the authors argue that on-policy RL implicitly regularizes the model towards KL-minimal solutions (with respect to the base model), allowing the model to find simpler solutions that are robust to catastrophic forgetting. Park et al. (2025) also report that SFT underperforms RL on the synthetic Countdown task; moreover, RL-only post-training can induce fundamentally improved OOD generalization. In addition, Chen et al. (2025) show that SFT can elicit pseudo-reasoning paths in large vision-language models by

only superficially imitating expert models. Such paths often contain uninformative or incorrect reasoning steps, and even hurt subsequent RL training stages. They also propose an RL-based approach which leads to more adaptive reasoning behavior.

Reasoning as graph search. Pathfinding in graphs has been widely used as a sandbox for understanding LLM reasoning capabilities. Abbe et al. (2024) propose the notion of globality degree to capture transformer learning ability, and show that regular transformers cannot efficiently solve cycle tasks. Sanford et al. (2024) study the ability of transformer networks to solve various graph algorithms in terms of their expressivity such as network width and depth. From an empirical perspective, synthetic graph pathfinding tasks have been used to study compositional reasoning ability (Khona et al., 2024) and understand internal prediction mechanisms (Cohen et al., 2025). For instance, Mirtaheiri et al. (2025) study the bridge graph to understand the benefits and tradeoffs with respect to parallel and single-context reasoning. Additionally, the path-star graph (which our construction generalizes) has been shown to be difficult to solve for LLMs, and has been used to study the limitations of reasoners trained via next-token prediction (Bachmann & Nagarajan, 2025; Frydenlund, 2024). Going beyond just graph search, several works have also analyzed the bigram policy class as a tractable way to give mathematical guarantees on reasoning ability (Nichani et al., 2024; Bu et al., 2026; Wang et al., 2026; Yang et al., 2025a).

3. Problem Formulation

3.1. CoT as a pathfinding task

We model our reasoning task as pathfinding on a toy graph that represents the world model (Sanford et al., 2024; Bachmann & Nagarajan, 2025; Kim et al., 2025). Specifically, we consider a multigraph consisting of the following components; see Figure 1.

- Source node s_0 , fork node f ;
- Diamond $\diamond(u, v)$, which is a subgraph of L undirected multiedges between nodes u, v ;
- Leaf nodes t_i .

The topology of the graph is as follows: the source node s_0 is connected to a fork node f through a directed edge to f (this will be the only directed edge in the graph). f now branches out in W directions. In each branch $i = 1, \dots, W$, we have a sequence of K diamonds:

$$\diamond(u_{i,1,l}, u_{i,1,r}), \diamond(u_{i,2,l}, u_{i,2,r}), \dots, \diamond(u_{i,K,l}, u_{i,K,r}),$$

where consecutive left and right diamonds are connected. When multiedges occur, we will denote the edges by $u \xleftrightarrow{j} v$

for $1 \leq j \leq L$ for the multiedges in $\diamond(u, v)$. Finally, we connect node $u_{i,K,r}$ to a leaf node t_i . For ease of notation, we will denote $u_{i,0,r} := f$ and $u_{i,K+1,l} := t_i$ for all i .

Reasoning task. In our setup, the reasoning model is prompted with a desired target node u , and the goal is to output a valid path from s to u . We are interested in understanding how post-training a language model with the standard paradigm of supervised fine-tuning (SFT) versus reinforcement learning with verifiable rewards (RLVR) influences the test-time behavior of the model, when prompted with a new reasoning task.

3.2. Pretraining

We consider a bigram model, in which each state consists of an edge and traversal direction (*not* the underlying vertices); that is, for an edge $e := u \leftrightarrow v$ in the graph, $u \rightarrow v$ and $v \rightarrow u$ are valid states. Furthermore, valid state transitions from an edge $u \rightarrow v$ would be to states in the set $N_v := \{v \rightarrow w : v \leftrightarrow w \in E\}$ (in particular, note that multiedges are distinct). For the model, we encode each state x as a one-hot vector in $\mathbb{R}^{2|E|}$, and we use a single-layer softmax predictor as follows:

$$\pi_{\Theta}(\cdot|x) = \text{softmax}(\langle \Theta, x \rangle), \quad \Theta \in (\mathbb{R} \cup \{-\infty\})^{2|E| \times 2|E|}.$$

We remark that this model is strictly more expressive than a *trigram model over nodes*: any trigram $\varphi(a|b, c)$ can be encoded as $\pi_{\Theta}(b \leftrightarrow c|a \leftrightarrow b)$, and moreover φ cannot express multiedges.

Initially, the predictor learns the world model or underlying graph, which we view as the pretrained model; this viewpoint has been used to study pathfinding tasks by Kim et al. (2025). For a given edge $u \rightarrow v$, we set the transition probabilities to be $1/|N_v|$ to edges in N_v , and 0 otherwise. When Θ is clear from context, we will often omit the subscript and denote the policy simply as π . We will also denote the underlying distribution of the described world model as $\mathcal{D}$, from which we draw samples (s, a) for current edge state and next edge state pairs such that the marginal probability over s is uniform over all edge states.

Theorem 1 (Warm-up: convergence of pretraining). *Suppose we train on the pairs $(s, a) \sim \mathcal{D}$ of current edge state and next edge state for our bigram such that $\mathbb{P}_{(s,a) \sim \mathcal{D}}[s]$ is uniform for all states. Under the loss*

$$L_{\text{pre}}(\Theta) := \mathbb{E}_{(s,a) \sim \mathcal{D}}[-\log \pi_{\Theta}(a|s)],$$

it holds that standard gradient flow from zero initialization learns transition probabilities in finite time.

The proof of Theorem 1 is deferred to Section A of the appendix. This justifies the following assumption on exactness of pretraining:

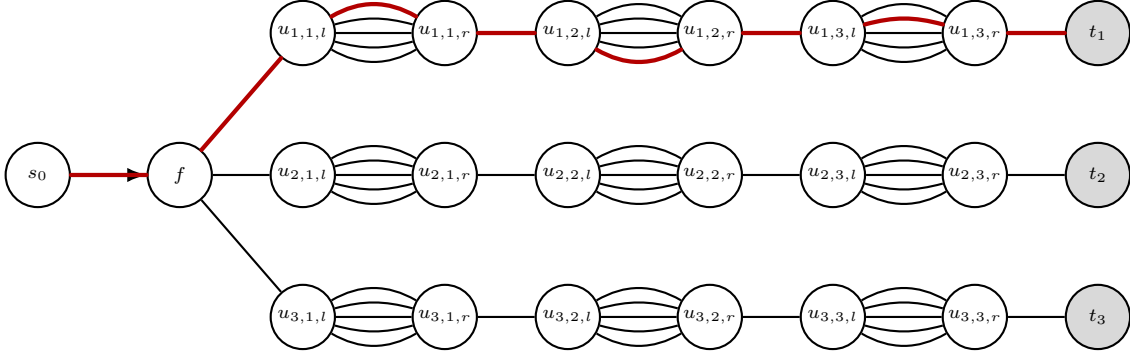


Figure 1. Structure of the world model graph, consisting of a source s_0 , a fork f , and W branches. Each branch contains a sequence of K diamonds, each with L multiedges, leading to a final leaf node. Here, $K = W = 3$ and $L = 5$. An example shortest-length path from s_0 to t_1 is highlighted in red.

Assumption 1. The pretrained model $\pi_{\Theta_{\text{pre}}}$ has converged arbitrarily close to the world model’s distribution.

This simplification is also used in Kim et al. (2025) to focus on analysis of post-training. Hence, $\pi_{\Theta_{\text{pre}}}$ can also be seen as a trigram over nodes, with multiedges taken into account.

3.3. Post-training

We study post-training π_{Θ} with either SFT or RLVR. In both setups, we consider training on data corresponding to path-target pairs, where the targets are sampled from the W leaf nodes t_i . However, the main difference is that SFT is given these pairs, whereas RLVR must obtain them via on-policy rollouts. We detail these methods in the following sections. In both cases, we assume the generation does not depend on the prompted target. That is, it is a pure bigram or Markov chain with initial state $s \rightarrow f$ and transition probabilities $\pi_{\Theta}(\cdot)$.

3.3.1. SFT

For SFT, we assume the model has access to golden shortest-length path examples, with no backtracking or exploration data. In practice, this corresponds to (for instance) feeding the model with gold standard solutions to a math problem. In particular, we train on shortest-length paths from the source s_0 to one of the W leaf nodes t_i for $1 \leq i \leq W$. These golden paths have the following characteristics: (1) it chooses the correct branch at the fork node f , and (2) for each diamond $\diamond(u_{i,j,l}, u_{i,j,r})$ for $1 \leq j \leq K$, it chooses exactly one of the L multiedges to traverse.

We optimize the cross-entropy loss over the dataset of golden paths and targets:

$$\min_{\Theta} L(\Theta) = \mathbb{E}_{x \sim \mathcal{U}(t_i), y \sim \mathcal{D}_x} \left[- \sum_{t=0}^{|y|-1} \log \pi_{\Theta}(y_{t+1} | y_t) \right], \quad (1)$$

where $\mathcal{D}_x$ is any (fully supported) distribution over golden shortest-length paths from the source to the target x . For ease of exposition, we consider optimizing this loss function via gradient flow, so that

$$\frac{d\Theta_{s,a}}{dt} = - \frac{\partial L}{\partial \Theta_{s,a}},$$

for all pairs s, a of current and next states, respectively.

3.3.2. RLVR

For RLVR, we generate on-policy rollouts starting from the source s_0 , with a verifier which checks whether the target node t_i has been reached. As with many open-source models (Kimi et al., 2025; DeepSeek-AI et al., 2025b), we will employ the use of a length penalty along with the verifier outcome reward, which for pathfinding is simply the length of the rollout. That is, our loss function is

$$\max_{\Theta} J(\Theta) = \mathbb{E}_{x \sim \mathcal{U}(t_i), y \sim \pi_{\Theta}} [r(x, y)], \quad (2)$$

$$r(x, y) = \mathbf{1}\{y \text{ hits node } x\} - \beta|y|,$$

with an appropriately chosen length penalty β . Alternatively, we could choose to use a finite horizon instead of a length penalty and heuristically expect the same results.

For a given rollout with target $x = t_i$, we assume that the verifier stops the rollout as soon as the current state s becomes $u_{i,K,r} \rightarrow x$. Then with a slight abuse of notation, $\mathbb{E}_{y \sim \pi}[|y|]$ represents the expected hitting time of t_i (which is the same for all targets by symmetry).

Policy gradient update. For the dynamical analysis, we consider the policy gradient update (Sutton et al., 1999):

$$\begin{aligned} \nabla_{\Theta} J(\Theta) &= \mathbb{E}_x \mathbb{E}_{y \sim \pi} [r(x, y) \nabla_{\Theta} \log \pi_{\Theta}(y)] \\ &= \mathbb{E}_x \mathbb{E}_{y \sim \pi} \left[r(x, y) \sum_{t=0}^{|y|-1} \nabla_{\Theta} \log \pi_{\Theta}(s_{t+1} | s_t) \right], \end{aligned}$$

where s_t is the current edge state, s_{t+1} is a potential next edge state, and the policy does not depend on the target.

Without loss of generality, we will work with the case where the length penalty strength $\beta = 1$. For simplicity of analysis, we consider policy gradient optimization via signed gradient flow. That is,

$$\frac{d\Theta_{s,a}}{dt} = \text{sgn}\left(\frac{\partial J}{\partial \Theta_{s,a}}\right),$$

for all pairs s, a of current and next state, respectively. Signed gradient descent has also been studied in Kim et al. (2025) to avoid fringe non-convergence issues.

4. Main Results

4.1. Notation

To rigorously analyze the two algorithms, we first observe that by symmetry of the population gradient and the graph topology, the transition probabilities for "topologically equivalent" edge states on different branches (i.e., at the same depth and same orientation with respect to the fork) are identical. Thus for any given j , the transition probabilities at any time $t \geq 0$ are the same for the following state types:

1. Forward (resp. backward) diamond connector states $u_{i,j-1,r} \rightarrow u_{i,j,l}$ (resp. $u_{i,j,l} \rightarrow u_{i,j-1,r}$) for all $1 \leq i \leq W$.
2. Forward (resp. backward) diamond multiedge states $u_{i,j,l} \xrightarrow{(\ell)} u_{i,j,r}$ (resp. $u_{i,j,r} \xrightarrow{(\ell)} u_{i,j,l}$) for all $1 \leq i \leq W$ and $1 \leq \ell \leq L$.

Due to multiedge states being identical logit-wise regardless of the choice of target, we will treat those states as identical; transitions into those states will have equal probability by symmetry. Hence, we denote a single multiedge state $u_{i,j,l} \rightarrow u_{i,j,r}$ to be the aggregate of $u_{i,j,l} \xrightarrow{(\ell)} u_{i,j,r}$, and similarly for $u_{i,j,r} \rightarrow u_{i,j,l}$.

We also use the following notation for states; see Figure 2.

1. $R_{i,j}^+$: state $u_{i,j,l} \rightarrow u_{i,j,r}$ (arrived at the right diamond node from the left). The probability of the forward connector (same direction as edge state) is denoted a_j , and the reverse multiedges back across the diamond have total probability $1 - a_j$ ($\frac{1-a_j}{L}$ per edge).
2. $R_{i,j}^-$: state $u_{i,j+1,l} \rightarrow u_{i,j,r}$ (arrived at the right diamond node from the right). The total probability of the backward multiedges is b_j ($\frac{b_j}{L}$ per edge), and the state back across the same edge has probability $1 - b_j$.

3. $L_{i,j}^+$: state $u_{i,j-1,r} \rightarrow u_{i,j,l}$ (arrived at the left diamond node from the left). The total probability of the forward multiedges is c_j , and the state back across the same edge has probability $1 - c_j$.
4. $L_{i,j}^-$: state $u_{i,j,r} \rightarrow u_{i,j,l}$ (arrived at the left diamond node from the right). The probability of the backward connector is d_j , and the multiedge states back across the diamond have total probability $1 - d_j$.

For the four types of states, we will essentially have two types of transitions: going forwards and going backwards. Note that over all i , the states of the same type and of same depth will have the same transition probabilities. Therefore, when the context is clear, we will often suppress the subscript i in the notation, and analyze the two cases of when i is the target branch versus when i is not the target branch. Finally, there are two special states which never have their logits updated:

1. $u_{i,K,r} \rightarrow t_i$: leaf-incoming states for $1 \leq i \leq W$. Since t_i is only adjacent to the node $u_{i,K,r}$, the state $u_{i,K,r} \rightarrow t_i$ deterministically transitions to $t_i \rightarrow u_{i,K,r}$ (unless t_i is the target, in which case the verifier stops the RL model rollout).
2. f_i : fork-incoming states $u_{i,1,l} \rightarrow f$ for $1 \leq i \leq W$. We fix the transition probabilities of these states to be uniform over the W edge states $f \rightarrow u_{i,1,l}$, including the same branch it came from.¹

We call the next edge states that the transition probabilities a_j, b_j, c_j, d_j above correspond to **desired** states; the other actions are called **undesired** states.

In particular, we have the following:

$$\begin{aligned} a_j &= \frac{\exp(\Theta_{R_j^+, L_{j+1}^+})}{\exp(\Theta_{R_j^+, L_{j+1}^+}) + L \exp(\Theta_{R_j^+, L_j^-})}, \\ b_j &= \frac{L \exp(\Theta_{R_j^-, L_j^-})}{L \exp(\Theta_{R_j^-, L_j^-}) + \exp(\Theta_{R_j^-, L_{j+1}^+})}, \\ c_j &= \frac{L \exp(\Theta_{L_j^+, R_j^+})}{L \exp(\Theta_{L_j^+, R_j^+}) + \exp(\Theta_{L_j^+, R_{j-1}^-})}, \\ d_j &= \frac{\exp(\Theta_{L_j^-, R_{j-1}^-})}{\exp(\Theta_{L_j^-, R_{j-1}^-}) + L \exp(\Theta_{L_j^-, R_j^+})}. \end{aligned}$$

¹If these logits are also updated, it can be shown that they will converge to the uniform distribution on the remaining $W - 1$ branches, discarding the branch it has exited from. Since this does not affect the final asymptotic guarantees, we fix these logits for simplicity.

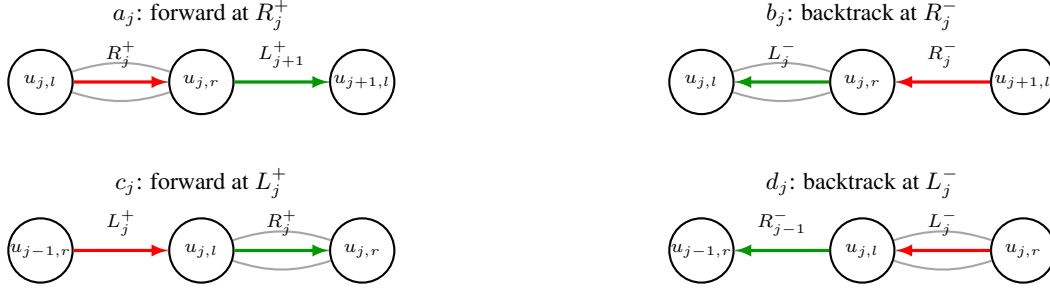


Figure 2. Desired edge-state transition types a_j, b_j, c_j, d_j . The current edge-state is denoted **red**; the desired next edge-state is denoted **green**.

For a given state s , let a_1 be its desired next state, and a_0 be its undesired next state. Then the pretrained model satisfies $\Theta_{s,a_1} = \Theta_{s,a_0} = 0$ and $\Theta_{s,a'} = -\infty$ for a' not in the set of possible next actions, so that $a_j(0) = d_j(0) = \frac{1}{L+1}$ and $b_j(0) = c_j(0) = \frac{L}{L+1}$.

Finally, we denote $H_f(t) = \mathbb{E}_{y \sim \pi_{\Theta(t)}}[|y|]$ to be the expected hitting time at time t after the start of post-training. Note that for any target $x = t_i$, this value is again the same.

4.2. SFT Learned Model

Theorem 2 (Transitions learned by SFT). *When training with gradient flow on the SFT loss (Equation (1)) over only golden paths, it holds that a_j, c_j converge arbitrarily close to 1 in finite time. On the other hand, b_j, d_j are fixed at $\frac{L}{L+1}, \frac{1}{L+1}$, respectively, over all time.*

Essentially, this theorem states that all forward-pointing edges will learn to put transition mass towards the next forward edge. On the other hand, because the model is only given golden paths towards the target, all backward pointing edge states remain uniform over its neighbors. In particular, because there is no explicit backtracking data in the fine-tuning dataset, the only ability to backtrack comes from what is learned during the pretraining phase, with no chance to be amplified post-training. We defer the full proof to Section A.

4.3. RLVR Learned Model

Theorem 3 (Transitions learned by RLVR). *When training with sign gradient flow on the RLVR loss (Equation (2)), a_j, b_j, c_j, d_j converge arbitrarily close to 1 in finite time.*

Intuitively, on-policy rollouts in RLVR penalizes the model for failed attempts containing long back-and-forth paths; it therefore needs to amplify its backtracking ability to mitigate this. As such, the learned model converges to the state where all edge states that point forward on a branch will learn to keep going forward, and all edge states that point backwards on a branch will learn to keep going backward. We defer the full proof of this theorem to Section B.

4.4. Separation in Inference-time Efficiency

Given the policy learned from either SFT or RLVR, our main result shows that the former can be exponentially more inefficient at inference time. This is due to the fact that fine-tuning on only the golden paths induces a lack of backtracking ability for the SFT model, which is well known to memorize the data distribution. For ease of exposition, we suppose that the SFT and RLVR models have converged to the transitions described in Theorem 2 and Theorem 3 (equivalently, the $t = \infty$ limit of the training process). In Section C, we prove the following theorem, which is our main result that the previous sections build up towards.

Theorem 4 (Inference-time separation). *Suppose that the learned RLVR model with $a_j, b_j, c_j, d_j = 1$ and the learned SFT model are prompted with any target node u in the graph to find a path starting from the source. Then the RLVR model requires $\Theta(WK)$ time in expectation to find u , while the SFT model requires $\Theta(WL^K)$ time.*

The above theorem implies a separation of $\Theta(L^K)$, which is exponential in the world model’s depth.

In addition to the above separation result, it is also insightful to understand what happens in the case where we allow additional inference time compute to orchestrate the search, say using some search agent. Suppose the search agent is such that it keeps track of which edge states have been visited at inference time, and it will prevent the policy from entering a visited edge state. Intuitively, in the case of the RLVR-learned policy, there would be essentially no impact, since inside a given branch each directed edge visited will only be visited once (this is a property of the learned policy), and the agent’s use would be purely in preventing it from visiting previously entered branches. For the SFT-learned policy, however, there will be an exponential improvement in inference-time efficiency, though not enough to match the RLVR policy. This is formalized in the following.

Corollary 1. *Given access to a search agent that prevents a given directed edge state from being visited twice during a single inference call, the RLVR model requires $\Theta(WK)$ time in expectation to find a target node u , while the SFT*

model requires $\Theta(WKL)$.

Proof. To formalize the intuition above, note that the SFT policy will still take $\Theta(K)$ time to reach a leaf of a given branch (since it goes forward with probability one). In a non-target branch, we claim that it will take $\Theta(KL)$ time to exit. This is because at the state R_j^- , we know that it will already have visited L_{j+1}^+ in the past; hence, it will take $\Theta(L)$ generations before it will continue to go backwards (e.g. $\Theta(L)$ rejected actions of going forwards by the search agent). Since there are W branches, we accrue a factor of W as well. $\square$

Indeed, there is still a $\Theta(L)$ factor separation between RLVR trained with backtracking against SFT on golden paths. Nevertheless, the fact that the search agent can allow an exponential speedup for some policies (albeit in this toy setting) demonstrates its power. Recent works have explored this axis of scaling test-time compute in many ways (Yao et al., 2024; Dang et al., 2026), and it is an open question to develop better orchestration and inference pipelines.

4.5. Distilling RLVR Reasoning Traces

Finally, we highlight when SFT can succeed for our reasoning task. Suppose we have access to the reasoning traces of a trained model, such as the outputs of a powerful closed-source model. If we fine-tune a pretrained base model on such traces (a process known as distillation (Shridhar et al., 2022)), we obtain similar guarantees in terms of inference-time efficiency, thereby avoiding the exponential blowup at inference time in Theorem 4.

Theorem 5 (SFT on reasoning traces). *Denote the joint distribution of target and reasoning trace pairs generated by the RLVR converged model to be $\mathcal{D}$. Then, if we fine-tune the pretrained model on these traces by optimizing the following loss:*

$$\min_{\Theta} L_{\text{distill}}(\Theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[- \sum_{t=0}^{|y|-1} \log \pi_{\Theta}(y_{t+1} | y_t) \right]$$

we can achieve $\Theta(WK)$ expected inference time compute.

The fact that reasoning traces learned by a reinforcement fine-tuned model are useful for distillation demonstrates the importance of backtracking in fine-tuning data.

5. Overview of Proofs

5.1. Proof Sketch of SFT Training

We essentially consider initialization from the pretrained model and analyze the gradient update on the cross-entropy loss. Since only the logits corresponding to a_j, c_j change

(as the training data does not contain any states R_j^- or L_j^-), we can reduce the argument into an analysis of an ODE on the logit gaps, and this is sufficient to prove Theorem 2.

5.2. Proof Sketch of RLVR Training

In contrast to the SFT setting, the dynamical analysis for RLVR is significantly more involved as the interaction between rollouts and backtracking comes into play. For each logit, the policy gradient is equivalent to

$$\frac{\partial J}{\partial \Theta_{s,a}} = \mathbb{E}_x[d_x(s)\pi(a|s)A_x(s,a)],$$

where $d_x(s) := \mathbb{E}_y[\sum_{t < \tau} \mathbf{1}\{s_t = s\}]$ is the expected number of times we reach state s before hitting the target x , and $A_x(s,a)$ is the advantage for the policy π for the reward (hitting time) when choosing action a compared to the other actions at state s .

In our analysis of RLVR, this also means that we will treat the previously defined quantity $d_x(\cdot)$ for such states as:

$$d_x(u_{i,j,l} \rightarrow u_{i,j,r}) := \sum_{\ell=1}^L \mathbb{E}_y \left[\sum_{t < \tau} \mathbf{1}\{s_t = (u_{i,j,l} \xrightarrow{(\ell)} u_{i,j,r})\} \right].$$

The analogous statement holds true for the reverse multi-edges as well.

For a fixed target x , define the hitting time $\tau_x := \min\{t \geq 0 : \text{head}(s_t) = x\}$. Also, let $h_x(s) := \mathbb{E}[\tau_x | s_0 = s]$, with absorption $h_x(s) = 0$ if $\text{head}(s) = x$ and the Bellman equation:

$$h_x(s) = 1 + \sum_a \pi(a|s)h_x(a) \quad (\text{head}(s) \neq x).$$

Then the advantage satisfies:

$$A_x(s,a) = \bar{h}_x(s) - h_x(a), \quad \bar{h}_x(s) := \sum_{a'} \pi(a'|s)h_x(a').$$

Also, recall the expected number of visits to a directed-edge state s before hitting x is

$$d_x(s) := \mathbb{E} \left[\sum_{t < \tau_x} \mathbf{1}\{s_t = s\} \right].$$

For multiedge states, we will treat this quantity as an aggregate of all L multiedges. Now, the update for each logit becomes:

$$\begin{aligned} \frac{\partial J}{\partial \Theta_{s,a}} &= \mathbb{E}_x[d_x(s)\pi(a|s)A_x(s,a)] \\ &= \frac{1}{W} \sum_{i=1}^W d_{t_i}(s)\pi(a|s)(\bar{h}_{t_i}(s) - h_{t_i}(a)). \end{aligned}$$

In each type of state, there is a desired action and an undesired action. Our goal is to show that the learned model converges to the state where all edge states that point forward on a branch will learn to keep going forward, and all edge states that point backwards on a branch will learn to keep going backward. The update rule of the logit difference of these two actions are related via the following lemma, which is proved in B.1.

Lemma 1. *For a state s , let $\Theta_{s,1}$ correspond to the logit for one of the desired next edge states, and $\Theta_{s,0}$ correspond to the logit for one of the undesired next edge states, and let the logit gap $\mathcal{D}_s := \Theta_{s,1} - \Theta_{s,0}$. Then*

$$\frac{d\mathcal{D}_s}{dt} = \text{sgn}(\mathbb{E}_x[d_x(s)(h_x(a_0) - h_x(a_1))]).$$

As a corollary, we can express the updates of the logit differences as follows, where we suppress the subscript i for brevity. The following hold for the four types of states:

1. R_j^+ state: Denote the logit gap as $\mathcal{D}_j^{(a)}$. Then,

$$\frac{d\mathcal{D}_j^{(a)}}{dt} = \text{sgn} \underbrace{\mathbb{E}_x[d_x(R_j^+) \cdot (h_x(L_j^-) - h_x(L_{j+1}^+))]}_{=:G_j^{(a)}}.$$

2. R_j^- state: Denote the logit gap as $\mathcal{D}_j^{(b)}$. Then,

$$\frac{d\mathcal{D}_j^{(b)}}{dt} = \text{sgn} \underbrace{\mathbb{E}_x[d_x(R_j^-) \cdot (h_x(L_{j+1}^+) - h_x(L_j^-))]}_{=:G_j^{(b)}}.$$

3. L_j^+ state: Denote the logit gap as $\mathcal{D}_j^{(c)}$. Then,

$$\frac{d\mathcal{D}_j^{(c)}}{dt} = \text{sgn} \underbrace{\mathbb{E}_x[d_x(L_j^+) \cdot (h_x(R_{j-1}^-) - h_x(R_j^+))]}_{=:G_j^{(c)}}.$$

4. L_j^- state: Denote the logit gap as $\mathcal{D}_j^{(d)}$. Then,

$$\frac{d\mathcal{D}_j^{(d)}}{dt} = \text{sgn} \underbrace{\mathbb{E}_x[d_x(L_j^-) \cdot (h_x(R_j^+) - h_x(R_{j-1}^-))]}_{=:G_j^{(d)}}.$$

Note that the dynamics depend exactly on the signs of $G_j^{(p)}$ for $p \in \{a, b, c, d\}$. We divide our analysis into two phases: 1) the time until all of the logit gaps are increasing (i.e., $G_j^{(p)} > 0$ for all $1 \leq j \leq K$ and $p \in \{a, b, c, d\}$) and 2) convergence from said time towards 1.

RLVR at post-training initialization. Denote the start of RLVR as time $t = 0$. The following lemma characterizes the properties of the gradients at initialization.

Lemma 2. *At $t = 0$, it holds that $G_j^{(a)}, G_j^{(c)} > 0$. However, $G_j^{(b)}, G_j^{(d)}$ may be negative for depths $1 \leq l(W, K, L) \leq j \leq r(W, K, L) \leq K - 1$, where $l(\cdot), r(\cdot)$ depend on the specific values of W, K, L .*

The full proof is deferred to B.2. Intuitively, this can be interpreted as the following. First, forward-oriented states (corresponding to $G_j^{(a)}, G_j^{(c)}$) want to continue forwards, as going backwards would be a waste of movement. Second, backward-oriented states (corresponding to $G_j^{(b)}, G_j^{(d)}$) near the end of a branch want to continue backwards, because they tend to believe that they came from a non-target leaf on the current branch. Third, backward-oriented states near the fork tend to put higher preference on the additional optionality of moving towards the fork as opposed to continuing forward on an uncertain branch. Finally, backward-oriented states in the middle are more "confused," in the sense that it is unclear if they are facing backwards because of backtracking or because of a U-turn from a forward-oriented state; this confusion should eventually be resolved as forward-oriented states converge to putting all mass forwards.

RLVR Phase I. We begin by showing the forward states' logits are increasing over all time; that is, we show that $G_j^{(a)}, G_j^{(c)}$ are always positive. As such, we will prove that once all the $G_j^{(b)}, G_j^{(d)}$ become positive, all four of these quantities will be increasing (and analyzed in Phase II). Hence, we will bound the time T_{meet} that it takes us to reach Phase II. This is given by the following lemma.

Lemma 3. *There exists a time T_{meet} such that for all $1 \leq j \leq K$ and all $p \in \{a, b, c, d\}$, $G_j^{(p)}(T_{\text{meet}}) > 0$.*

The proof of this fact relies on analyzing how $G_j^{(p)}$ evolves over this time. In particular, we can show theoretically and empirically that the middle interval of depths for which $G_j^{(b)}$ and $G_j^{(d)}$ are negative becomes shorter and shorter in Phase I, which is at the core of the proof of Lemma 3. Full details are deferred to the appendix.

RLVR Phase II. Continuing on, we enter the second phase of the training dynamics after time T_{meet} . In the setting of Lemma 3, we now consider the additional time needed after time $t = T_{\text{meet}}$ to reach the regime where $\min_j (\min\{a_j, b_j, c_j, d_j\}) \geq 1 - \kappa$ for some $\kappa \ll 1$, for which we prove the following guarantee.

Lemma 4. *For any $\kappa \in (0, 1)$, there exists a time T' such that at time $t = T_{\text{meet}} + T'$, the following holds:*

$$\min\{a_j(t), b_j(t), c_j(t), d_j(t)\} \geq 1 - \kappa \quad \forall 1 \leq j \leq K.$$

The result of this phase follows from the self-reinforcing nature of the dynamics in this regime. Combining the results thus far gives Theorem 3. Full proofs can be found in Section B in the appendix.

5.3. Proof Sketch of Inference Time Separation

Given the construction of the converged model in Theorem 3, the model takes $\Theta(K)$ time to traverse any branch back-and-forth, and traverses W branches on expectation before hitting the target branch. Therefore, the required inference time compute is $\Theta(WK)$ for the RLVR model.

For the converged SFT model in Theorem 2, the backtracking behavior is more complex. Upon entering a branch i , since $a_j = c_j = 1$ for all j , we reach the t_i node in $\Theta(K)$ time. However, exiting the branch is more complicated. Starting from the state R_{K+1}^+ with head t_i , let g_i denote the expected time of first entry into R_{K+1-i}^- , and f_i denote the expected time of first entry into L_{K+1-i}^- . Then, we have the following recursion with base case $g_1 = 1$:

$$f_i = \frac{L+1}{L}g_i + \frac{2i-1}{L} + 1,$$

$$g_i = (L+1)f_{i-1} + (2i-2)L + 1.$$

We prove and analyze this recurrence in Section C of the appendix. In particular, the time needed from entry of a wrong branch to getting back to the fork state is $g_{K+1} = O(L^K)$, leading to a total compute of $\Theta(WL^K)$.

6. Experiments

In this section, we run experiments on our synthetic setup to validate our theory. We provide example plots for the case of $W = 15, K = 15, L = 5$, run under sign gradient descent with learning rate 0.01. Figure 3 shows that the final hitting time indeed converges to $\Theta(KW)$, as suggested by 4.4. Moreover, Figure 4 shows that all probabilities a_j, b_j, c_j, d_j indeed converge to 1 with a rate (only) depending on their type. While not visually clear, we note that there is a very brief regime in the beginning in which the probabilities b and d are decreasing for middle depth nodes.

7. Conclusion

We highlight backtracking as a key capability for reasoning. We introduce a simple graph pathfinding model of chain-of-thought reasoning and give a dynamical comparison between supervised fine-tuning on shortest-path demonstrations and RL with verifiable rewards. We show that SFT on gold traces need not update backward-state transitions and can incur exponential search cost, whereas RLVR’s on-policy rollouts provide signal to learn efficient backtracking from failed paths, yielding a provable inference-time compute separation. Finally, we show that RLVR-generated

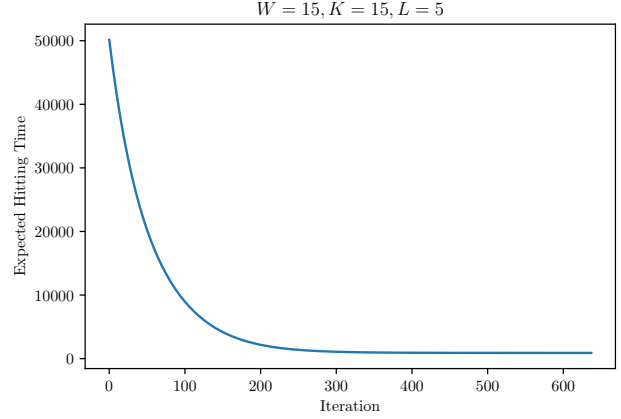


Figure 3. Expected hitting time of RLVR-trained model against training iterations. Here, hitting time converges to $4WK = 900$.

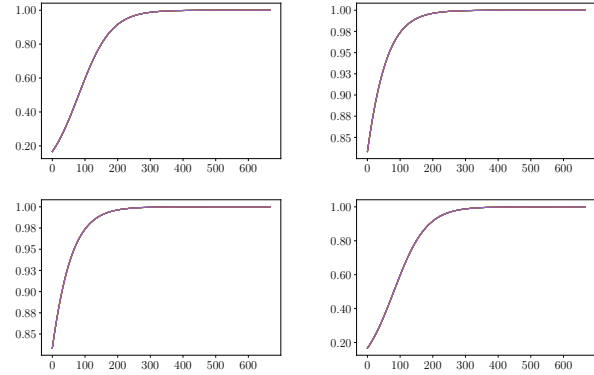


Figure 4. a_j, b_j, c_j, d_j values for RLVR-trained model (ordered in Z shape). Values of edges over all depths j are overlaid. Note that $a(0) = d(0) = \frac{1}{L+1}$ and $b(0) = c(0) = \frac{L}{L+1}$.

traces can be distilled via supervised learning to transfer efficient backtracking to a base model.

Acknowledgements

SW acknowledges support from an NSF Graduate Research Fellowship.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Abbe, E., Bengio, S., Lotfi, A., Sandon, C., and Saremi, O. How far can transformers reason? The locality barrier and

- inductive scratchpad. In *Advances in Neural Information Processing Systems*, 2024.
- Bachmann, G. and Nagarajan, V. The pitfalls of next-token prediction, 2025. URL <https://arxiv.org/abs/2403.06963>.
- Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., et al. Graph of thoughts: solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- Bu, D., Huang, W., Han, A., Nitanda, A., Xue, B., Zhang, Q., Wong, H.-S., and Suzuki, T. Distributional biases in post-training: A markovian analysis of reasoning trajectories, 2026. URL <https://arxiv.org/abs/2511.07368>.
- Cai, H. J., Wang, J., Chen, X., and Dhingra, B. How much backtracking is enough? exploring the interplay of sft and rl in enhancing llm reasoning, 2025. URL <https://arxiv.org/abs/2505.24273>.
- Chen, H., Tu, H., Wang, F., Liu, H., Tang, X., Du, X., Zhou, Y., and Xie, C. SFT or RL? An Early Investigation into Training RL-Like Reasoning Large Vision-Language Models. *arXiv preprint arXiv:2504.11468*, 2025.
- Christiano, P., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences, 2023. URL <https://arxiv.org/abs/1706.03741>.
- Chu, T., Zhai, Y., Yang, J., Tong, S., Xie, S., Schuurmans, D., Le, Q. V., Levine, S., and Ma, Y. SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training. *arXiv preprint arXiv:2501.17161*, 2025a.
- Chu, T., Zhai, Y., Yang, J., Tong, S., Xie, S., Schuurmans, D., Le, Q. V., Levine, S., and Ma, Y. Sft memorizes, rl generalizes: A comparative study of foundation model post-training, 2025b. URL <https://arxiv.org/abs/2501.17161>.
- Cohen, A., Gromov, A., Yang, K., and Tian, Y. Spectral journey: How transformers predict the shortest path, 2025. URL <https://arxiv.org/abs/2502.08794>.
- Dang, X., Agarwal, R., Porto, R., Goyal, A., Fowl, L. H., and Arora, S. Escaping the cognitive well: Efficient competition math with off-the-shelf models, 2026. URL <https://arxiv.org/abs/2602.16793>.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning, 2025a. URL <https://arxiv.org/abs/2501.12948>.
- DeepSeek-AI, Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., Lu, C., Zhao, C., Deng, C., Xu, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Li, E., Zhou, F., Lin, F., Dai, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Li, H., Liang, H., Wei, H., Zhang, H., Luo, H., Ji, H., Ding, H., Tang, H., Cao, H., Gao, H., Qu, H., Zeng, H., Huang, J., Li, J., Xu, J., Hu, J., Chen, J., Xiang, J., Yuan, J., Cheng, J., Zhu, J., Ran, J., Jiang, J., Qiu, J., Li, J., Song, J., Dong, K., Gao, K., Guan, K., Huang, K., Zhou, K., Huang, K., Yu, K., Wang, L., Zhang, L., Wang, L., Zhao, L., Yin, L., Guo, L., Luo, L., Ma, L., Wang, L., Zhang, L., Di, M. S., Xu, M. Y., Zhang, M., Zhang, M., Tang, M., Zhou, M., Huang, P., Cong, P., Wang, P., Wang, Q., Zhu, Q., Li, Q., Chen, Q., Du, Q., Xu, R., Ge, R., Zhang, R., Pan, R., Wang, R., Yin, R., Xu, R., Shen, R., Zhang, R., Liu, S. H., Lu, S., Zhou, S., Chen, S., Cai, S., Chen, S., Hu, S., Liu, S., Hu, S., Ma, S., Wang, S., Yu, S., Zhou, S., Pan, S., Zhou, S., Ni, T., Yun, T., Pei, T., Ye, T., Yue, T., Zeng, W., Liu, W., Liang, W., Pang, W., Luo, W., Gao,

- W., Zhang, W., Gao, X., Wang, X., Bi, X., Liu, X., Wang, X., Chen, X., Zhang, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Li, X., Yang, X., Li, X., Chen, X., Su, X., Pan, X., Lin, X., Fu, X., Wang, Y. Q., Zhang, Y., Xu, Y., Ma, Y., Li, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Qian, Y., Yu, Y., Zhang, Y., Ding, Y., Shi, Y., Xiong, Y., He, Y., Zhou, Y., Zhong, Y., Piao, Y., Wang, Y., Chen, Y., Tan, Y., Wei, Y., Ma, Y., Liu, Y., Yang, Y., Guo, Y., Wu, Y., Wu, Y., Cheng, Y., Ou, Y., Xu, Y., Wang, Y., Gong, Y., Wu, Y., Zou, Y., Li, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Zhao, Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Huang, Z., Wu, Z., Li, Z., Zhang, Z., Xu, Z., Wang, Z., Gu, Z., Zhu, Z., Li, Z., Zhang, Z., Xie, Z., Gao, Z., Pan, Z., Yao, Z., Feng, B., Li, H., Cai, J. L., Ni, J., Xu, L., Li, M., Tian, N., Chen, R. J., Jin, R. L., Li, S. S., Zhou, S., Sun, T., Li, X. Q., Jin, X., Shen, X., Chen, X., Song, X., Zhou, X., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Huang, Z., Xu, Z., Zhang, Z., Ji, D., Liang, J., Guo, J., Chen, J., Xia, L., Wang, M., Li, M., Zhang, P., Chen, R., Sun, S., Wu, S., Ye, S., Wang, T., Xiao, W. L., An, W., Wang, X., Sun, X., Wang, X., Tang, Y., Zha, Y., Zhang, Z., Ju, Z., Zhang, Z., and Qu, Z. Deepseek-v3.2: Pushing the frontier of open large language models, 2025b. URL <https://arxiv.org/abs/2512.02556>.
- Frydenlund, A. The mystery of the pathological path-star task for language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 12493–12516. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.695. URL <http://dx.doi.org/10.18653/v1/2024.emnlp-main.695>.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Khona, M., Okawa, M., Hula, J., Ramesh, R., Nishi, K., Dick, R., Lubana, E. S., and Tanaka, H. Towards an understanding of stepwise inference in transformers: A synthetic graph navigation model, 2024. URL <https://arxiv.org/abs/2402.07757>.
- Kim, J., Wu, D., Lee, J., and Suzuki, T. Metastable Dynamics of Chain-of-Thought Reasoning: Provable Benefits of Search, RL and Distillation. *arXiv preprint arXiv:2502.01694*, 2025.
- Kimi, T., Du, A., Gao, B., Xing, B., Jiang, C., Chen, C., Li, C., Xiao, C., Du, C., Liao, C., et al. Kimi k1.5: scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.
- Kumar, K., Ashraf, T., Thawakar, O., Anwer, R. M., Cholakal, H., Shah, M., Yang, M.-H., Torr, P. H. S., Khan, F. S., and Khan, S. Llm post-training: A deep dive into reasoning large language models, 2025. URL <https://arxiv.org/abs/2502.21321>.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Long, J. Large language model guided tree-of-thought, 2023. URL <https://arxiv.org/abs/2305.08291>.
- Mirtaheri, P., Edelman, E., Jelassi, S., Malach, E., and Boix-Adsera, E. Let Me Think! A Long Chain-of-Thought Can Be Worth Exponentially Many Short Ones. *arXiv preprint arXiv:2505.21825*, 2025.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. sl: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- Nichani, E., Damian, A., and Lee, J. D. How transformers learn causal structure with gradient descent. *arXiv preprint arXiv:2402.14735*, 2024.
- OpenAI. Openai o1 system card, 2024. URL <https://arxiv.org/abs/2412.16720>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- Park, S., Kaur, S., and Arora, S. How does rl post-training induce skill composition? a case study on countdown, 2025. URL <https://arxiv.org/abs/2512.01775>.
- Qin, T., Alvarez-Melis, D., Jelassi, S., and Malach, E. To backtrack or not to backtrack: When sequential search limits model reasoning, 2025. URL <https://arxiv.org/abs/2504.07052>.
- Rohatgi, D., Shetty, A., Saleh, D., Li, Y., Moitra, A., Risteski, A., and Foster, D. J. Taming imperfect process verifiers: A sampling perspective on backtracking, 2025. URL <https://arxiv.org/abs/2510.03149>.
- Sanford, C., Fatemi, B., Hall, E., Tsitsulin, A., Kazemi, M., Halcrow, J., Perozzi, B., and Mirrokni, V. Understanding transformer reasoning capabilities via graph algorithms. *arXiv preprint arXiv:2405.18512*, 2024.

- Shalev-Shwartz, S. and Shashua, A. From reasoning to super-intelligence: A search-theoretic perspective, 2025. URL <https://arxiv.org/abs/2507.15865>.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shenfeld, I., Pari, J., and Agrawal, P. RL’s razor: Why online reinforcement learning forgets less, 2025. URL <https://arxiv.org/abs/2509.04259>.
- Shridhar, K., Stolfo, A., and Sachan, M. Distilling reasoning capabilities into smaller language models. *arXiv preprint arXiv:2212.00193*, 2022.
- Singh, A., Arora, K., Keh, S., Mercat, J., Hashimoto, T., Finn, C., and Kumar, A. Improving the efficiency of test-time search in LLMs with backtracking, 2025. URL <https://openreview.net/forum?id=hJ2BCYGvFg>.
- Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In Solla, S., Leen, T., and Müller, K. (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.
- Wang, Y., Yang, Q., Zeng, Z., Ren, L., Liu, L., Peng, B., Cheng, H., He, X., Wang, K., Gao, J., Chen, W., Wang, S., Du, S. S., and Shen, Y. Reinforcement learning for reasoning in large language models with one training example, 2025. URL <https://arxiv.org/abs/2504.20571>.
- Wang, Z., Dong, Y., and Lei, Q. When does chain-of-thought help: A markovian perspective, 2026. URL <https://arxiv.org/abs/2603.00306>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Wen, X., Liu, Z., Zheng, S., Ye, S., Wu, Z., Wang, Y., Xu, Z., Liang, X., Li, J., Miao, Z., Bian, J., and Yang, M. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms, 2025. URL <https://arxiv.org/abs/2506.14245>.
- Xu, F., Hao, Q., Zong, Z., Wang, J., Zhang, Y., Wang, J., Lan, X., Gong, J., Ouyang, T., Meng, F., Shao, C., Yan, Y., Yang, Q., Song, Y., Ren, S., Hu, X., Li, Y., Feng, J., Gao, C., and Li, Y. Towards large reasoning models: A survey of reinforced reasoning with large language models, 2025. URL <https://arxiv.org/abs/2501.09686>.
- Yang, W., Liao, M., and Fan, K. Markov chain of thought for efficient mathematical reasoning, 2025a. URL <https://arxiv.org/abs/2410.17635>.
- Yang, X.-W., Zhu, X.-Y., Wei, W.-D., Zhang, D.-C., Shao, J.-J., Zhou, Z., Guo, L.-Z., and Li, Y.-F. Step back to leap forward: Self-backtracking for boosting reasoning of language models, 2025b. URL <https://arxiv.org/abs/2502.04404>.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL <https://arxiv.org/abs/2305.10601>.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- Zhu, X., Xia, M., Wei, Z., Chen, W.-L., Chen, D., and Meng, Y. The surprising effectiveness of negative reinforcement in llm reasoning, 2025. URL <https://arxiv.org/abs/2506.01347>.

A. Pretraining and Supervised Fine-Tuning

We start off with the following general lemma for cross-entropy loss.

Lemma 5. *Let $\mathcal{Q}$ be any distribution over pairs (s, a) of current edge-state s and next edge-state a . Define the marginal $d_{\mathcal{Q}}(s) := \mathbb{P}_{(s,a) \sim \mathcal{Q}}[s]$ and the conditional $p_{\mathcal{Q}}(a | s) := \mathbb{P}_{(s,a) \sim \mathcal{Q}}[a | s]$. Consider the cross-entropy objective*

$$L_{\mathcal{Q}}(\Theta) := \mathbb{E}_{(s,a) \sim \mathcal{Q}}[-\log \pi_{\Theta}(a | s)].$$

Then for every pair (s, a) ,

$$\frac{\partial L_{\mathcal{Q}}}{\partial \Theta_{s,a}} = d_{\mathcal{Q}}(s) \left(\pi_{\Theta}(a | s) - p_{\mathcal{Q}}(a | s) \right).$$

In particular, under gradient flow $\frac{d\Theta_{s,a}}{dt} = -\partial L_{\mathcal{Q}} / \partial \Theta_{s,a}$, the dynamics of each row $\Theta_{s,\cdot}$ depends only on transitions out of s that appear in the bigram distribution; for instance, if $d_{\mathcal{Q}}(s) = 0$ then $\Theta_{s,\cdot}(t)$ remains constant.

Proof. We rewrite the loss as follows:

$$L_{\mathcal{Q}}(\Theta) = \sum_{s'} d_{\mathcal{Q}}(s') \sum_b p_{\mathcal{Q}}(b | s') [-\log \pi_{\Theta}(b | s')].$$

In particular, for a given state s , we have

$$\frac{\partial}{\partial \Theta_{s,a}} \log \pi_{\Theta}(b | s') = \mathbf{1}\{s' = s\} (\mathbf{1}\{b = a\} - \pi_{\Theta}(a | s)).$$

Therefore

$$\begin{aligned} \frac{\partial L_{\mathcal{Q}}}{\partial \Theta_{s,a}} &= -d_{\mathcal{Q}}(s) \sum_b p_{\mathcal{Q}}(b | s) (\mathbf{1}\{b = a\} - \pi_{\Theta}(a | s)) \\ &= d_{\mathcal{Q}}(s) (\pi_{\Theta}(a | s) - p_{\mathcal{Q}}(a | s)), \end{aligned}$$

as desired. $\square$

We now use Lemma 5 to prove the pretraining convergence result.

Proof of Theorem 1. We first initialize $\Theta(0) = 0$ to be the zero logit matrix. Let $\mathcal{D}$ be the world model transition distribution described in the main text. For each state $s = u \rightarrow v$, let $\mathcal{A}(s) := N_v$ denote the set of valid next edge-states and let $m_s := |\mathcal{A}(s)|$. Under $\mathcal{D}$, we have the conditional target distribution

$$p_{\mathcal{D}}(a | s) = \begin{cases} \frac{1}{m_s}, & a \in \mathcal{A}(s), \\ 0, & a \notin \mathcal{A}(s). \end{cases}$$

By Lemma 5, the pretraining gradient flow satisfies, for every s, a ,

$$\frac{d\Theta_{s,a}}{dt} = -d_{\mathcal{D}}(s) (\pi_{\Theta(t)}(a | s) - p_{\mathcal{D}}(a | s)).$$

By assumption, we have that all of the states of the world model have equally likely marginal probability. Hence, for all of the gradient flow logits, we can rescale time by $d_{\mathcal{D}}(s)$, so that:

$$\frac{d\Theta_{s,a}}{dt} = -(\pi_{\Theta(t)}(a | s) - p_{\mathcal{D}}(a | s)).$$

We now fix any state s . Because $\Theta_{s,a}(0) = 0$ for all a , we have complete symmetry over all states at $t = 0$. Moreover, the target distribution $p_{\mathcal{D}}(\cdot | s)$ is invariant under permutations within the two groups $\mathcal{A}(s)$ and $\mathcal{A}(s)^c$. Since the ODE is

deterministic and coordinates within each group have identical right-hand sides whenever they are equal, uniqueness of ODE solutions implies that for all $t \geq 0$,

$$\Theta_{s,a}(t) = u_s(t) \quad \forall a \in \mathcal{A}(s), \quad \Theta_{s,a}(t) = v_s(t) \quad \forall a \notin \mathcal{A}(s),$$

for some scalars $u_s(t), v_s(t)$.

Let $M := 2|E|$ be the total number of edge-states, and denote $n_s := M - m_s$. Define the total probability mass assigned to valid actions

$$p_s(t) := \sum_{a \in \mathcal{A}(s)} \pi_{\Theta(t)}(a | s) = \frac{m_s e^{u_s(t)}}{m_s e^{u_s(t)} + n_s e^{v_s(t)}}.$$

Then each valid action has probability $p_s(t)/m_s$ and each invalid action has probability $(1 - p_s(t))/n_s$. Applying Lemma 5 to one representative valid action and one representative invalid action gives

$$\frac{du_s}{dt} = \left(\frac{1}{m_s} - \frac{p_s(t)}{m_s} \right) = \frac{1 - p_s(t)}{m_s}, \quad \frac{dv_s}{dt} = -\frac{1 - p_s(t)}{n_s}.$$

Hence for the logit gap $g_s(t) := u_s(t) - v_s(t)$ we have

$$\frac{dg_s}{dt} = (1 - p_s(t)) \left(\frac{1}{m_s} + \frac{1}{n_s} \right) \geq 0,$$

so $g_s(t)$ is nondecreasing. Writing $w_s(t) := e^{g_s(t)}$ and using

$$p_s(t) = \frac{m_s w_s(t)}{m_s w_s(t) + n_s} \quad \Rightarrow \quad 1 - p_s(t) = \frac{n_s}{m_s w_s(t) + n_s},$$

we obtain

$$\frac{dw_s}{dt} = w_s(t) \frac{dg_s}{dt} = \left(\frac{1}{m_s} + \frac{1}{n_s} \right) \frac{n_s w_s(t)}{m_s w_s(t) + n_s} \geq \frac{1}{m_s},$$

where for the final inequality we have used that $w_s(t) \geq w_s(0) = 1$ and thus $\frac{n_s w_s}{m_s w_s + n_s} \geq \frac{n_s}{m_s + n_s}$. Therefore $w_s(t) \geq 1 + \frac{1}{m_s} t$ and hence

$$1 - p_s(t) = \frac{n_s}{m_s w_s(t) + n_s} \leq \frac{n_s}{m_s \left(1 + \frac{1}{m_s} t \right)} = \frac{n_s}{m_s + t}.$$

Altogether, we have that the total invalid mass $1 - p_s(t)$ goes to 0 as $t \rightarrow \infty$, and the valid mass goes to 1. Because all valid logits remain equal, the distribution over valid actions is uniform at all times:

$$\pi_{\Theta(t)}(a | s) = \frac{p_s(t)}{m_s} \quad \forall a \in \mathcal{A}(s).$$

Consequently, for any $\varepsilon > 0$ we may choose $T_s(\varepsilon) := n_s/\varepsilon$ so that for all $t \geq T_s(\varepsilon)$ we have $1 - p_s(t) \leq \varepsilon$, and therefore

$$\max_{a \in \mathcal{A}(s)} \left| \pi_{\Theta(t)}(a | s) - \frac{1}{m_s} \right| \leq \frac{\varepsilon}{m_s} \quad \text{and} \quad \sum_{a \notin \mathcal{A}(s)} \pi_{\Theta(t)}(a | s) \leq \varepsilon.$$

Since this holds for every state s , pretraining under gradient flow learns the world model transition kernel up to arbitrary error in finite time, as desired. $\square$

Using a similar technique of analyzing a training with a teacher distribution using cross-entropy loss, we can prove the convergence of the SFT model. In the following section, we provide the proof for Theorem 2.

Proof of Theorem 2. We assume the SFT phase is initialized at the pretrained world model described in the text, i.e., for every state s all valid next-actions have equal logits, and invalid actions have zero probability (equivalently logits $-\infty$). We also assume $\mathcal{D}_x$ is uniform with respect to the targets; this only affects the relative timescales of convergence.

Let $\mathcal{Q}_{\text{SFT}}$ denote the induced distribution over adjacent transition pairs (s, a) obtained by the following procedure: sample a target $x \sim \mathcal{U}(\{t_i\})$, sample a golden path $y \sim \mathcal{D}_x$, and then sample a uniformly random transition (y_{t-1}, y_t) (i.e. each edge connecting diamonds, and each of the L multiedges at a diamond will be sampled uniformly at random).

Then the SFT objective can be written as

$$L_{\text{SFT}}(\Theta) = \mathbb{E}_{(s,a) \sim \mathcal{Q}_{\text{SFT}}} [-\log \pi_{\Theta}(a \mid s)].$$

By Lemma 5, for each state s the gradient flow depends only on the conditional $p_{\mathcal{Q}_{\text{SFT}}}(\cdot \mid s)$, and if a state s is never encountered in golden paths then its row never updates.

First, observe that by construction, every golden path is a shortest path from s_0 to some t_i and therefore never contains backtracking. In particular, none of the backward-pointing states $R_{i,j}^-$ or $L_{i,j}^-$ appear in y . Hence for each such backward state s we have $d_{\mathcal{Q}_{\text{SFT}}}(s) = 0$, and Lemma 5 implies $\Theta_{s,\cdot}(t)$ is constant over time. Therefore the associated transition probabilities remain equal to their pretrained values:

$$b_j(t) = b_j(0) = \frac{L}{L+1}, \quad d_j(t) = d_j(0) = \frac{1}{L+1}.$$

We now analyze the forward-oriented states. First, consider a forward-oriented state $R_{i,j}^+$, which has exactly one forward action and L backward actions. On any golden path, whenever we are in state $R_{i,j}^+$ the next state is deterministically the forward connector state $L_{i,j+1}^+$. Therefore the SFT conditional satisfies $p_{\mathcal{Q}_{\text{SFT}}}(\cdot \mid R_{i,j}^+)$ being a point mass on the forward connector action.

By symmetry among the L backward multiedges and the symmetric pretrained initialization, their logits remain equal for all time, so the row can be summarized by two scalars: $u(t)$ (logit for the unique forward connector) and $v(t)$ (common logit for each backward multiedge). Then

$$a_j(t) = \frac{e^{u(t)}}{e^{u(t)} + L e^{v(t)}}.$$

Applying Lemma 5 to this row yields (up to time rescaling of $d_{\mathcal{Q}_{\text{SFT}}}(R_{i,j}^+)$):

$$\frac{du}{dt} = 1 - a_j(t), \quad \frac{dv}{dt} = -\frac{1 - a_j(t)}{L}.$$

Hence for the logit gap $g(t) := u(t) - v(t)$ we have

$$\frac{dg}{dt} = (1 - a_j(t)) \left(1 + \frac{1}{L}\right) \geq 0.$$

Let $w(t) := e^{g(t)}$. Using $a_j(t) = \frac{w(t)}{w(t)+L}$ one computes

$$\frac{dw}{dt} = w(t) \frac{dg}{dt} = (L+1) \frac{w(t)}{w(t)+L} \geq 1,$$

since $w(t) \geq w(0) = 1$. Therefore $w(t) \geq 1+t$ and

$$1 - a_j(t) = \frac{L}{w(t)+L} \leq \frac{L}{t+L+1}.$$

Thus for any $\kappa > 0$, taking $t \geq \frac{L}{\kappa} - L - 1$ ensures $a_j(t) \geq 1 - \kappa$.

Finally, we consider a forward-oriented state of type $L_{i,j}^+$. This state has L forward multiedges transitions and one backward transition. On any golden path, whenever we are at $L_{i,j}^+$ the next action is always one of the forward multiedges across $\diamond(u_{i,j,l}, u_{i,j,r})$. Therefore the SFT conditional satisfies $p_{\mathcal{Q}_{\text{SFT}}}(\cdot \mid L_{i,j}^+)$ having zero probability on the backward transition.

Let $\rho_j(t)$ denote the model's probability of taking the backward connector at $L_{i,j}^+$; then $c_j(t) = 1 - \rho_j(t)$. The backward connector logit is pushed down at rate proportional to $\rho_j(t)$, and hence $\rho_j(t) \rightarrow 0$ and $c_j(t) \rightarrow 1$. Since $\mathcal{D}_x$ is invariant

under permutations of the L multiedges in each diamond (i.e. each forward multiedge is equally likely in the population loss), we similarly have that all L forward multiedges share a common logit $u(t)$ and the backward connector has logit $v(t)$, so

$$c_j(t) = \frac{Le^{u(t)}}{Le^{u(t)} + e^{v(t)}} = \frac{Lw(t)}{Lw(t) + 1}, \quad w(t) := e^{u(t)-v(t)}.$$

The same calculation gives $\frac{dw}{dt} \geq 1/L$, hence $w(t) \geq 1 + t/L$ and

$$1 - c_j(t) = \frac{1}{Lw(t) + 1} \leq \frac{1}{t + L + 1}.$$

Thus for any $\kappa > 0$, taking $t \geq \frac{1}{\kappa} - L - 1$ ensures $c_j(t) \geq 1 - \kappa$, as desired. $\square$

B. RLVR

B.1. Setup and Notations

Recall that:

$$\nabla_{\Theta} J(\Theta) = \mathbb{E}_x \mathbb{E}_{y \sim \pi} [r(x, y) \nabla_{\Theta} \log \pi_{\Theta}(y)] = \mathbb{E}_x \mathbb{E}_{y \sim \pi_{\Theta}} \left[r(x, y) \sum_{t=0}^{|y|-1} \nabla_{\Theta} \log \pi_{\Theta}(s_{t+1} | s_t) \right].$$

In the main text, we claimed that this is equivalent to:

$$\frac{\partial J}{\partial \Theta_{s,a}} = \mathbb{E}_x [d_x(s) \pi(a|s) A_x(s, a)]$$

where $d_x(s) := \mathbb{E}_y [\sum_{t < \tau} \mathbf{1}\{s_t = s\}]$ is the expected number of times we reach state s before hitting the target x , and $A_x(s, a)$ is the advantage for the policy π for the reward (hitting time) when choosing action a compared to the other actions at state s . We prove this claim below.

Lemma 6. *The policy gradient can be expressed as:*

$$\mathbb{E}_x \mathbb{E}_{y \sim \pi} \left[r(x, y) \sum_{t=0}^{|y|-1} \nabla_{\Theta} \log \pi(s_{t+1} | s_t) \right] = \mathbb{E}_x [d_x(s) \pi(a|s) A_x(s, a)],$$

where

$$A_x(s, a) = \bar{h}_x(s) - h_x(a) \quad \text{where} \quad \bar{h}_x(s) := \sum_{a'} \pi(a'|s) h_x(a').$$

Here, we recall that $h_x(\cdot)$ denotes the expected hitting time of a state whose head is the target x , starting from a given current state.

Proof. First, we note that under softmax parameterization, we have

$$\pi_{\Theta}(a|s) = \frac{\exp(\Theta_{s,a})}{\sum_{a'} \exp(\Theta_{s,a'})},$$

and so

$$\frac{\partial \log \pi_{\Theta}(a_t | s_t)}{\partial \Theta_{s,a}} = \mathbf{1}\{s_t = s\} (\mathbf{1}\{a_t = a\} - \pi_{\Theta}(a|s)).$$

In our case, we have that $r(x, y) = 1 - |y|$. We can first simplify by noting that for any fixed t ,

$$\mathbb{E}[\nabla_{\Theta} \log \pi_{\Theta}(a_t | s_t) | s_t] = \sum_a \pi_{\Theta}(a | s_t) \nabla_{\Theta}(a | s_t) = \nabla_{\Theta} \left(\sum_a \pi_{\Theta}(a | s_t) \right) = 0.$$

Therefore, we obtain:

$$\begin{aligned}\nabla_{\Theta} J(\Theta) &= \mathbb{E}_{x,y} \left[\sum_{t=0}^{|y|-1} (1 - |y|) \mathbf{1}\{s_t = s\} (\mathbf{1}\{a_t = a\} - \pi_{\Theta}(a|s)) \right] \\ &= -\mathbb{E}_{x,y} \left[\sum_{t=0}^{|y|-1} |y| \mathbf{1}\{s_t = s\} (\mathbf{1}\{a_t = a\} - \pi_{\Theta}(a|s)) \right].\end{aligned}$$

To continue, we note that for any fixed t , it holds that:

$$\mathbb{E}[t \nabla_{\Theta} \log_{\Theta}(a|s_t) | s_t] = 0,$$

due to a similar argument as before (rewards from the past do not impact the gradient). Therefore, we can rewrite the policy gradient as:

$$\begin{aligned}\nabla_{\Theta} J(\Theta) &= -\mathbb{E}_{x,y} \left[\sum_{t=0}^{|y|-1} |y| \mathbf{1}\{s_t = s\} (\mathbf{1}\{a_t = a\} - \pi_{\Theta}(a|s)) \right] \\ &= -\mathbb{E}_{x,y} \left[\sum_{t=0}^{|y|-1} (|y| - t) \mathbf{1}\{s_t = s\} (\mathbf{1}\{a_t = a\} - \pi_{\Theta}(a|s)) \right] \\ &= \mathbb{E}_x [d_x(s) \pi(a|s) (\bar{h}_x(s) - h_x(a))],\end{aligned}$$

as desired. $\square$

We are now ready to prove Lemma 1.

Proof of Lemma 1. First, we note that $\pi(a_0|s) + \pi(a_1|s) = 1$. Then, we observe that:

$$\begin{aligned}\bar{h}_x(s) - h_x(a_1) &= \pi(a_0|s) h_x(a_0) + \pi(a_1|s) h_x(a_1) - h_x(a_1) \\ &= \pi(a_0|s) (h_x(a_0) - h_x(a_1)).\end{aligned}$$

Similarly,

$$\bar{h}_x(s) - h_x(a_0) = -\pi(a_1|s) (h_x(a_0) - h_x(a_1)).$$

Observe that $\bar{h}_x(s) - h_x(a_1)$ and $\bar{h}_x(s) - h_x(a_0)$ have opposite signs. Combining with Lemma 6, we obtain:

$$\begin{aligned}\frac{d\mathcal{D}_s}{dt} &= \text{sgn}(\mathbb{E}_x [d_x(s) [\pi(a_1|s) (\bar{h}_x(s) - h_x(a_1)) - \pi(a_0|s) (\bar{h}_x(s) - h_x(a_0))]]) \\ &= \text{sgn}(2\mathbb{E}_x [d_x(s) \pi(a_1|s) \pi(a_0|s) (h_x(a_0) - h_x(a_1))]) \\ &= \text{sgn}(\mathbb{E}_x [d_x(s) (h_x(a_0) - h_x(a_1))]),\end{aligned}$$

as desired. $\square$

To characterize the dynamics of RLVR, we first define the following notations.

Definition 1. Fix a target leaf x . We define the following quantities:

1. Let $H_f := h_x(u_{i,1,l} \rightarrow f)$ denote the hitting time of a fixed target x from any of the states whose head is the fork f (including the initial source state $s_0 \rightarrow f$); by symmetry they are all equal, hence why we consider a single value.
2. For state s not on the branch of x , we define $\tilde{g}(s)$ to be the expected time of hitting the fork, starting at state s . That is, $\tilde{g}(s) := \mathbb{E}[\tau_f]$ where $\tau_f := \min\{t \geq 0 : s_0 = s, \text{head}(s_t) = f\}$. In particular, this means that $h_x(s) = \tilde{g}(s) + H_f$, and we have the absorbing condition $h_x(u_{i,1,l} \rightarrow f) = 0$ for branch i not equal to the branch of target x .

3. For state s on the branch of x , we define $g(s)$ to be the expected time of hitting either f or x (e.g. both states are absorbing). That is, $g(s) := \mathbb{E}[\tau_{\{f,x\}}]$ where $\tau_{\{f,x\}} := \min\{t \geq 0 : s_0 = s, \text{head} \in \{f, x\}\}$.
4. For state s on the branch of x , we define $q(s)$ to be the probability it hits f before x . In particular, this means that $h_x(s) = g(s) + q(s)H_f$.

Lemma 7. Define the following q -gaps:

$$\delta_j := q(L_j^-) - q(L_{j+1}^+), \quad \epsilon_j := q(R_{j-1}^-) - q(R_j^+)$$

Then, it holds that $\delta_j > 0$ and $\epsilon_j > 0$ for all j .

Proof. For δ_j , this follows intuitively from the fact that all paths from L_{j+1}^+ to the fork must pass through L_j^- . Hence, the event that a path arrives back at the fork from L_{j+1}^+ is a subset of the event that a path arrives back to the fork from L_j^- . Similar reasoning holds for ϵ_j , and the lemma follows. $\square$

Lemma 8. Define the following $\tilde{g}$ -gaps:

$$\Delta_j := \tilde{g}(L_{j+1}^+) - \tilde{g}(L_j^-), \quad E_j := \tilde{g}(R_j^+) - \tilde{g}(R_{j-1}^-).$$

Then, it holds that $\Delta_j \geq 2$ and $E_j \geq 2$ for all j .

Proof. For Δ_j , this follows intuitively from the fact that all paths from L_{j+1}^+ to the fork must pass through L_j^- , which requires at least two moves. A similar reasoning holds for E_j , and the lemma follows. $\square$

We remark that we cannot immediately obtain clean positivity results for the g -gaps (i.e., the expected absorbing time onto $\{f, x\}$ on for states on the branch of a target x). To analyze it, we first define the following quantities.

Definition 2. Fix a target x , and consider the entry state L_1^+ upon entering any branch from the fork. We define the following for a state s on this same branch:

1. When s is on the target branch, define $\mu(s) := \mathbb{E}\left[\sum_{t < \tau_{\{f,x\}}} \mathbf{1}\{s_t = s\} | s_0 = L_1^+\right]$. In other words, μ_s is the expected number of visits to state s during a target branch attempt.
2. When s is not on the target branch, define $\tilde{\mu}(s) := \mathbb{E}\left[\sum_{t < \tau_f} \mathbf{1}\{s_t = s\} | s_0 = L_1^+\right]$. In other words, $\tilde{\mu}_s$ is the expected number of visits to state s during a non-target branch attempt until it goes back to f .

Definition 3. Fix a target x , and define the success probability of hitting the target upon entering its branch p_{succ} before hitting f . That is,

$$p_{\text{succ}} := \mathbb{P}\{\text{head}(s_{\tau_{\{f,x\}}}) = x | s_0 = L_1^+\},$$

where L_1^+ is the entry state of the branch of x .

Proposition 1. Fix a target leaf x . Then, the following hold:

1. If s is on the same branch as x , then we have $d_x(s) = \frac{\mu(s)}{p_{\text{succ}}}$.
2. If s is on a different branch from x , then we have $d_x(s) = \frac{\tilde{\mu}(s)}{p_{\text{succ}}}$.

Proof. Note that the probability of success for each branch entry is $\frac{p_{\text{succ}}}{W}$, as one needs to choose the target branch with probability $1/W$ and then succeed with probability p_{succ} . Hence, the total number of fork departures on expectation is W/p_{succ} , so the expected number of entries into any fixed branch is $1/p_{\text{succ}}$. $\square$

Proposition 2. Define:

$$\Delta g_j^{(a)} := g(L_j^-) - g(L_{j+1}^+), \quad \Delta g_j^{(c)} := g(R_{j-1}^-) - g(R_j^+).$$

Furthermore, define $\Delta g_j^{(b)} := -\Delta g_j^{(a)}$ and $\Delta g_j^{(d)} := -\Delta g_j^{(c)}$. Then, the following hold:

1. $G_j^{(a)} = \mathbb{E}_x[d_x(R_{i,j}^+) \cdot (h_x(L_{i,j}^-) - h_x(L_{i,j+1}^+))] = \frac{1}{W p_{\text{succ}}} \left[\mu(R_j^+) (\Delta g_j^{(a)} + \delta_j H_f) - (W-1) \tilde{\mu}(R_j^+) \Delta_j \right].$
2. $G_j^{(b)} = \mathbb{E}_x[d_x(R_{i,j}^-) \cdot (h_x(L_{i,j+1}^+) - h_x(L_{i,j}^-))] = \frac{1}{W p_{\text{succ}}} \left[\mu(R_j^-) (-\Delta g_j^{(a)} - \delta_j H_f) + (W-1) \tilde{\mu}(R_j^-) \Delta_j \right].$
3. $G_j^{(c)} = \mathbb{E}_x[d_x(L_{i,j}^+) \cdot (h_x(R_{i,j-1}^-) - h_x(R_{i,j}^+))] = \frac{1}{W p_{\text{succ}}} \left[\mu(L_j^+) (\Delta g_j^{(c)} + \epsilon_j H_f) - (W-1) \tilde{\mu}(L_j^+) E_j \right].$
4. $G_j^{(d)} = \mathbb{E}_x[d_x(L_{i,j}^-) \cdot (h_x(R_{i,j}^+) - h_x(R_{i,j-1}^-))] = \frac{1}{W p_{\text{succ}}} \left[\mu(L_j^-) (-\Delta g_j^{(c)} - \epsilon_j H_f) + (W-1) \tilde{\mu}(L_j^-) E_j \right].$

Proof. We will first consider the case $G_j^{(a)}$. If R_j^+ is on the target branch (which happens with probability $1/W$), then we have:

$$\begin{aligned} d_x(R_j^+) \cdot (h_x(L_{i,j}^-) - h_x(L_{i,j+1}^+)) &= \frac{\mu(R_j^+)}{p_{\text{succ}}} \cdot ((g(L_j^-) - g(L_{j+1}^+)) + (q(L_j^-) - q(L_{j+1}^+)) H_f) \\ &= \frac{\mu(R_j^+)}{p_{\text{succ}}} \cdot (\Delta g_j^{(a)} + \delta_j H_f). \end{aligned}$$

If R_j^+ is not on the target branch (which happens with probability $(W-1)/W$), then we have:

$$d_x(R_j^+) \cdot (h_x(L_{i,j}^-) - h_x(L_{i,j+1}^+)) = \frac{\tilde{\mu}(R_j^+)}{p_{\text{succ}}} \cdot (\tilde{g}(L_j^-) - \tilde{g}(L_{j+1}^+)) = \frac{\tilde{\mu}(R_j^+)}{p_{\text{succ}}} \cdot (-\Delta_j).$$

This gives the desired expectation. Analogous calculations can be done for the other three cases to obtain the same result. $\square$

We now observe that we can rescale time in the four logit ODE's by a factor of $\frac{2}{W p_{\text{succ}}}$, since those terms do not change the sign in our gradient flow. Therefore, we redefine:

1. $G_j^{(a)} := \left[\mu(R_j^+) (\Delta g_j^{(a)} + \delta_j H_f) - (W-1) \tilde{\mu}(R_j^+) \Delta_j \right].$
2. $G_j^{(b)} := \left[\mu(R_j^-) (-\Delta g_j^{(a)} - \delta_j H_f) + (W-1) \tilde{\mu}(R_j^-) \Delta_j \right].$
3. $G_j^{(c)} := \left[\mu(L_j^+) (\Delta g_j^{(c)} + \epsilon_j H_f) - (W-1) \tilde{\mu}(L_j^+) E_j \right].$
4. $G_j^{(d)} := \left[\mu(L_j^-) (-\Delta g_j^{(c)} - \epsilon_j H_f) + (W-1) \tilde{\mu}(L_j^-) E_j \right].$

so that for $p_j \in \{a_j, b_j, c_j, d_j\}$, the gradient flow is

$$\frac{d\mathcal{D}_j^{(p)}}{dt} = \text{sgn}(G_j^{(p)}).$$

B.2. Summary of Important Quantities

For ease of exposition, we summarize the notations defined above in a concise list with its correspondence to branch type.

- Off-target branch: $\tilde{\mu}(\cdot), \tilde{g}(\cdot), \Delta_j, E_j$
- On-target branch: $\mu(\cdot), g(\cdot), \Delta g_j^a, \Delta g_j^c, q(\cdot), \delta_j, \epsilon_j, p_{\text{succ}}$
- Overall: H_f , which is the expected hitting time of fixed target t_i from a branch. Since the targets t_i are symmetric, we have that $J(\Theta) = 1 - H_f$. Moreover, using Proposition 1 yields $H_f = \frac{W + g(L_1^+) + (W-1)\tilde{g}(L_1^+)}{p_{\text{succ}}}.$

Note that in the context of the population loss, the expectation is uniform over the choice of on-target branch. In the remainder of this section, we will calculate these quantities exactly in terms of arbitrary a_j, b_j, c_j, d_j .

We give closed form expressions for all of the below quantities, which were calculated with the assistance of SymPy. Then, in B.2.4, we verify separately that these quantities (which are by definition unique) indeed hold for the stochastic system we have defined thus far. Unless denoted otherwise, the quantities below hold for all $1 \leq j \leq K$, with the convention that a state R_0^- is a fork-head state, and L_{K+1}^+ is a leaf-head state, as well as the summation and product of an empty set being 0 and 1 respectively.

B.2.1. OFF-TARGET QUANTITIES

Define the quantity $r_j := \frac{a_j c_j}{b_j d_j}$.

1. $\tilde{\mu}(\cdot)$: We have

$$\begin{aligned}\tilde{\mu}(L_j^+) &= \prod_{m=1}^{j-1} r_m, \\ \tilde{\mu}(R_j^+) &= \tilde{\mu}(L_j^-) = \frac{c_j}{d_j} \prod_{m=1}^{j-1} r_m, \\ \tilde{\mu}(R_j^-) &= \tilde{\mu}(L_{j+1}^+) = \prod_{m=1}^j r_m.\end{aligned}$$

2. E_j : We have

$$E_j = \sum_{m=j}^K \left(\frac{2(a_m + b_m)}{b_m d_m} \prod_{i=j}^{m-1} \frac{a_i c_{i+1}}{b_i d_i} \right).$$

3. Δ_j : We have

$$\Delta_j = \frac{2 + c_{j+1} E_{j+1}}{b_j} = \frac{d_j E_j - 2}{a_j}.$$

4. $\tilde{g}(\cdot)$: Define $F_j := \tilde{g}(R_j^-)$, so that $F_0 = 0$. Then we have

$$F_j = \sum_{m=1}^j (2 + (1 - d_m) E_m + (1 - b_m) \Delta_m),$$

and

$$\begin{aligned}\tilde{g}(R_j^-) &= F_j, \\ \tilde{g}(R_j^+) &= F_{j-1} + E_j, \\ \tilde{g}(L_j^+) &= 1 + F_{j-1} + c_j E_j, \\ \tilde{g}(L_j^-) &= 1 + F_{j-1} + (1 - d_j) E_j.\end{aligned}$$

We will generally not be working with $\tilde{g}$ directly, but rather their adjacent differences E_j, Δ_j .

B.2.2. ON-TARGET QUANTITIES

Define the following quantities:

$$\begin{aligned}
 r_j &:= \frac{a_j c_j}{b_{j-1} d_j}, \quad \forall 2 \leq j \leq K, \\
 P_j &:= \prod_{m=j+1}^K r_m, \\
 \alpha_j &:= (1 - a_j) + \frac{a_j}{d_j} (1 - c_j), \\
 S_j &:= \sum_{m=j}^K \alpha_m P_m, \\
 Z_q &:= P_1 \left(1 - a_1 + \frac{a_1}{d_1} \right) + S_2.
 \end{aligned}$$

We also have the following quantities.

1. δ_j : We have $\delta_j = P_j / Z_q$.
2. ϵ_j : We have $\epsilon_j = \delta_j a_j / d_j$.
3. p_{succ} : We have

$$p_{\text{succ}} = \frac{a_1 c_1}{d_1} \frac{P_1}{Z_q}.$$

4. Δg_j^a : First, define the following:

$$\begin{aligned}
 \mathcal{B}_j &:= - \sum_{m=j+1}^K \frac{2(c_m + d_m)}{b_{m-1} d_m} \prod_{i=j+1}^{m-1} r_i, \\
 \beta_m &:= (1 - a_m) + \frac{a_m(1 - c_m)}{d_m}, \\
 \gamma_m &:= 2 - \frac{2(1 - c_m)}{d_m}, \\
 P_g &:= \sum_{m=2}^K \beta_m P_m, \\
 Q_g &:= \sum_{m=2}^K (\beta_m \mathcal{B}_m + \gamma_m).
 \end{aligned}$$

Then, it holds that:

$$\begin{aligned}
 \Delta g_K^a &= - \frac{\left(\frac{a_1}{d_1} + 1 - a_1 \right) \mathcal{B}_1 - \frac{2}{d_1} + Q_g + 1}{\left(\frac{a_1}{d_1} + 1 - a_1 \right) P_1 + P_g}, \\
 \Delta g_j^a &= P_j \Delta g_K^a + \mathcal{B}_j.
 \end{aligned}$$

5. Δg_j^c : We have

$$\Delta g_j^c = \frac{a_j \Delta g_j^a - 2}{d_j}.$$

Equivalently, when $2 \leq j \leq K$,

$$\Delta g_j^c = \frac{b_{j-1} \Delta g_{j-1}^a + 2}{c_j}.$$

6. $g(\cdot)$: For simplicity, we will not write down the full expression, since we will generally be working with adjacent differences Δg_j^a and Δg_j^c .
7. $\mu(\cdot)$: We define the prefix product $A_j := \prod_{m=1}^j a_m$, as well as B_j, C_j, D_j similarly. Then,

$$\begin{aligned} \mu(L_j^+) &= \frac{A_{j-1} C_{j-1}}{B_{j-1} D_{j-1}} \cdot \frac{S_j + \frac{a_j c_j}{d_j} P_j}{Z_q}, \\ \mu(R_j^+) &= \frac{A_{j-1} C_j}{B_{j-1} D_j} \cdot \frac{S_{j+1} + P_j}{Z_q}, \\ \mu(L_j^-) &= \mu(R_j^+) - p_{\text{succ}}, \\ \mu(R_j^-) &= \mu(L_{j+1}^+) - p_{\text{succ}}. \end{aligned}$$

In the last line, for $j = K$ we use the auxiliary convention $\mu(L_{K+1}^+) := p_{\text{succ}}$, so that $\mu(R_K^-) = 0$.

B.2.3. CLOSED FORMS FOR $G_j^{(p)}$

The aforementioned quantities are also sufficient for us to write a closed form for the rescaled $G_j^{(p)}$ for $p \in \{a, b, c, d\}$ used in the sign dynamics above. As before, define $A_j = \prod_{m=1}^j a_m$ to be the prefix product of a , and similarly for B_j, C_j, D_j . In particular, we have:

$$\begin{aligned} G_j^{(a)} &= \frac{A_{j-1} C_j}{B_{j-1} D_j} \cdot \frac{S_{j+1} + P_j}{Z_q} \left[P_j \Delta g_K^a + \mathcal{B}_j + \frac{P_j}{Z_q} H_f \right] - (W-1) \frac{A_{j-1} C_j}{B_{j-1} D_j} \Delta_j, \\ G_j^{(b)} &= \left[\frac{A_j C_j}{B_j D_j} \cdot \frac{S_{j+1} + b_j P_j}{Z_q} - p_{\text{succ}} \right] \cdot \left[-P_j \Delta g_K^a - \mathcal{B}_j - \frac{P_j}{Z_q} H_f \right] + (W-1) \frac{A_j C_j}{B_j D_j} \Delta_j, \\ G_j^{(c)} &= \left[\frac{A_{j-1} C_{j-1}}{B_{j-1} D_{j-1}} \cdot \frac{S_j + \frac{a_j c_j}{d_j} P_j}{Z_q} \right] \cdot \left[\frac{a_j (P_j \Delta g_K^a + \mathcal{B}_j) - 2}{d_j} + \frac{a_j}{d_j} \frac{P_j}{Z_q} H_f \right] - (W-1) \frac{A_{j-1} C_{j-1}}{B_{j-1} D_{j-1}} E_j, \\ G_j^{(d)} &= \left[\frac{A_{j-1} C_j}{B_{j-1} D_j} \cdot \frac{S_{j+1} + P_j}{Z_q} - p_{\text{succ}} \right] \cdot \left[-\frac{a_j (P_j \Delta g_K^a + \mathcal{B}_j) - 2}{d_j} - \frac{a_j}{d_j} \frac{P_j}{Z_q} H_f \right] + (W-1) \frac{A_{j-1} C_j}{B_{j-1} D_j} E_j. \end{aligned}$$

B.2.4. VERIFICATION OF THE CLOSED FORMS

We now verify that the expressions above solve the relevant linear systems. Since each system is finite and absorbing (i.e. from every transient state there is a fixed-length path to either the fork or the target with positive probability), we have that the Bellman and state transition equations have unique solutions. As such, we can verify the above closed form expressions via direct substitution and calculation.

We first list the equations to be checked. For off-target visit counts, the equations are

$$\begin{aligned} \tilde{\mu}(L_1^+) &= 1, \\ \tilde{\mu}(L_{j+1}^+) &= a_j \tilde{\mu}(R_j^+) + (1 - b_j) \tilde{\mu}(R_j^-), \\ \tilde{\mu}(R_j^+) &= c_j \tilde{\mu}(L_j^+) + (1 - d_j) \tilde{\mu}(L_j^-), \\ \tilde{\mu}(L_j^-) &= (1 - a_j) \tilde{\mu}(R_j^+) + b_j \tilde{\mu}(R_j^-), \\ \tilde{\mu}(R_j^-) &= (1 - c_{j+1}) \tilde{\mu}(L_{j+1}^+) + d_{j+1} \tilde{\mu}(L_{j+1}^-), \end{aligned}$$

where for $j = K$ we use the leaf convention that the nonexistent state has $\tilde{\mu}(L_{K+1}^-) = 0$ and L_{K+1}^+ transitions to R_K^- with

probability one. For off-target hitting times, the Bellman equations are

$$\begin{aligned}\tilde{g}(R_0^-) &= 0, & \tilde{g}(L_{K+1}^+) &= 1 + \tilde{g}(R_K^-), \\ \tilde{g}(L_j^+) &= 1 + c_j \tilde{g}(R_j^+) + (1 - c_j) \tilde{g}(R_{j-1}^-), \\ \tilde{g}(L_j^-) &= 1 + d_j \tilde{g}(R_{j-1}^-) + (1 - d_j) \tilde{g}(R_j^+), \\ \tilde{g}(R_j^+) &= 1 + a_j \tilde{g}(L_{j+1}^+) + (1 - a_j) \tilde{g}(L_j^-), \\ \tilde{g}(R_j^-) &= 1 + b_j \tilde{g}(L_j^-) + (1 - b_j) \tilde{g}(L_{j+1}^+).\end{aligned}$$

On the target branch, the Bellman equations for q have boundaries $q(R_0^-) = 1$ and $q(L_{K+1}^+) = 0$ and satisfy

$$\begin{aligned}q(L_j^+) &= c_j q(R_j^+) + (1 - c_j) q(R_{j-1}^-), \\ q(L_j^-) &= d_j q(R_{j-1}^-) + (1 - d_j) q(R_j^+), \\ q(R_j^+) &= a_j q(L_{j+1}^+) + (1 - a_j) q(L_j^-), \\ q(R_j^-) &= b_j q(L_j^-) + (1 - b_j) q(L_{j+1}^+).\end{aligned}$$

The Bellman equations for g have boundaries $g(R_0^-) = g(L_{K+1}^+) = 0$ and satisfy

$$\begin{aligned}g(L_j^+) &= 1 + c_j g(R_j^+) + (1 - c_j) g(R_{j-1}^-), \\ g(L_j^-) &= 1 + d_j g(R_{j-1}^-) + (1 - d_j) g(R_j^+), \\ g(R_j^+) &= 1 + a_j g(L_{j+1}^+) + (1 - a_j) g(L_j^-), \\ g(R_j^-) &= 1 + b_j g(L_j^-) + (1 - b_j) g(L_{j+1}^+).\end{aligned}$$

Finally, the target branch visit counts satisfy

$$\begin{aligned}\mu(L_1^+) &= 1, \\ \mu(L_{j+1}^+) &= a_j \mu(R_j^+) + (1 - b_j) \mu(R_j^-) \quad (j < K), \\ \mu(R_j^+) &= c_j \mu(L_j^+) + (1 - d_j) \mu(L_j^-), \\ \mu(L_j^-) &= (1 - a_j) \mu(R_j^+) + b_j \mu(R_j^-), \\ \mu(R_j^-) &= (1 - c_{j+1}) \mu(L_{j+1}^+) + d_{j+1} \mu(L_{j+1}^-).\end{aligned}$$

Lemma 9. *The closed forms for $\tilde{\mu}$ in B.2.1 solve the off-target Bellman equations.*

Proof. Let $p_j := \prod_{m=1}^{j-1} r_m$, so $p_1 = 1$ and $p_{j+1} = r_j p_j = \frac{a_j c_j}{b_j d_j} p_j$. The closed forms are

$$\tilde{\mu}(L_j^+) = p_j, \quad \tilde{\mu}(R_j^+) = \tilde{\mu}(L_j^-) = \frac{c_j}{d_j} p_j, \quad \tilde{\mu}(R_j^-) = \tilde{\mu}(L_{j+1}^+) = p_{j+1}.$$

For $j = 1$, it is straightforward to see $\tilde{\mu}(L_1^+) = p_1 = 1$. For the remaining equations,

$$\begin{aligned}a_j \tilde{\mu}(R_j^+) + (1 - b_j) \tilde{\mu}(R_j^-) &= \frac{a_j c_j}{d_j} p_j + (1 - b_j) p_{j+1} = p_{j+1}, \\ c_j \tilde{\mu}(L_j^+) + (1 - d_j) \tilde{\mu}(L_j^-) &= c_j p_j + (1 - d_j) \frac{c_j}{d_j} p_j = \frac{c_j}{d_j} p_j, \\ (1 - a_j) \tilde{\mu}(R_j^+) + b_j \tilde{\mu}(R_j^-) &= (1 - a_j) \frac{c_j}{d_j} p_j + b_j p_{j+1} = \frac{c_j}{d_j} p_j.\end{aligned}$$

Finally,

$$(1 - c_{j+1}) \tilde{\mu}(L_{j+1}^+) + d_{j+1} \tilde{\mu}(L_{j+1}^-) = p_{j+1},$$

where for $j < K$ this is $(1 - c_{j+1}) p_{j+1} + c_{j+1} p_{j+1} = p_{j+1}$, and for $j = K$ it follows from the leaf convention. $\square$

Lemma 10. *The closed forms for E_j, Δ_j, F_j and $\tilde{g}$ in B.2.1 solve the off-target Bellman equations.*

Proof. Using $E_{K+1} = 0$, the definition of E_j gives

$$\begin{aligned} E_j &= \frac{2(a_j + b_j)}{b_j d_j} + \frac{a_j c_{j+1}}{b_j d_j} E_{j+1}, \\ d_j E_j &= 2 + a_j \Delta_j, \quad b_j \Delta_j = 2 + c_{j+1} E_{j+1}. \end{aligned}$$

Moreover, $F_j - F_{j-1} = 2 + (1 - d_j)E_j + (1 - b_j)\Delta_j$. The boundary equations hold because $F_0 = 0$ and $\tilde{g}(L_{K+1}^+) = 1 + F_K = 1 + \tilde{g}(R_K^-)$.

For the Bellman equations of $\tilde{g}(L_j^+)$ and $\tilde{g}(L_j^-)$, we have:

$$\begin{aligned} 1 + c_j \tilde{g}(R_{j-1}^+) + (1 - c_j) \tilde{g}(R_{j-1}^-) &= 1 + c_j(F_{j-1} + E_j) + (1 - c_j)F_{j-1} \\ &= 1 + F_{j-1} + c_j E_j = \tilde{g}(L_j^+), \\ 1 + d_j \tilde{g}(R_{j-1}^-) + (1 - d_j) \tilde{g}(R_j^+) &= 1 + d_j F_{j-1} + (1 - d_j)(F_{j-1} + E_j) \\ &= 1 + F_{j-1} + (1 - d_j)E_j = \tilde{g}(L_j^-). \end{aligned}$$

For R_j^+ , substituting the closed forms and the above identities yields:

$$\begin{aligned} 1 + a_j \tilde{g}(L_{j+1}^+) + (1 - a_j) \tilde{g}(L_j^-) \\ &= 1 + a_j(1 + F_j + c_{j+1} E_{j+1}) + (1 - a_j)(1 + F_{j-1} + (1 - d_j)E_j) \\ &= F_{j-1} + 2 + (1 - d_j)E_j + a_j \Delta_j = F_{j-1} + E_j = \tilde{g}(R_j^+). \end{aligned}$$

Finally, for R_j^- ,

$$\begin{aligned} 1 + b_j \tilde{g}(L_j^-) + (1 - b_j) \tilde{g}(L_{j+1}^+) \\ &= 1 + b_j(1 + F_{j-1} + (1 - d_j)E_j) + (1 - b_j)(1 + F_j + c_{j+1} E_{j+1}) \\ &= F_j + (1 - b_j)(2 - b_j \Delta_j + c_{j+1} E_{j+1}) = F_j = \tilde{g}(R_j^-). \end{aligned}$$

□

Lemma 11. *The closed forms for δ_j , ϵ_j and p_{succ} in B.2.2 satisfy the Bellman equations for q .*

Proof. We claim that the Bellman equations for q imply the following equivalent equations for the gaps:

$$\begin{aligned} \epsilon_j &= \frac{a_j}{d_j} \delta_j, \\ \delta_{j-1} &= r_j \delta_j \quad (2 \leq j \leq K), \\ 1 &= \left(1 - a_1 + \frac{a_1}{d_1}\right) \delta_1 + \sum_{m=2}^K \alpha_m \delta_m. \end{aligned}$$

Indeed, the first identity follows by subtracting the R_j^+ equation from the L_j^- equation. The second follows by subtracting the L_j^+ equation from the R_{j-1}^- equation and using the first identity. For the third and final identity, we use the following telescoping decomposition:

$$\begin{aligned} 1 &= q(R_0^-) - q(L_{K+1}^+) \\ &= \epsilon_1 + (q(R_1^+) - q(L_2^+)) + \sum_{m=2}^K (q(L_m^+) - q(L_{m+1}^+)) \\ &= \left(\frac{a_1}{d_1} + 1 - a_1\right) \delta_1 + \sum_{m=2}^K \alpha_m \delta_m. \end{aligned}$$

We now substitute $\delta_j = P_j/Z_q$ and $\epsilon_j = a_j\delta_j/d_j$. Since $P_{j-1} = r_jP_j$, the recurrence holds. The normalization holds by the definition of Z_q . Finally,

$$p_{\text{succ}} = 1 - q(L_1^+) = q(R_0^-) - q(L_1^+) = c_1\epsilon_1 = \frac{a_1c_1}{d_1} \frac{P_1}{Z_q}.$$

□

Lemma 12. *The closed forms for Δg_j^a and Δg_j^c in B.2.2 satisfy the Bellman system for the target branch absorption times g .*

Proof. Let $U_j := \Delta g_j^a$ and $V_j := \Delta g_j^c$. Subtracting the target branch Bellman equations gives the equivalent gap system

$$\begin{aligned} V_j &= \frac{a_j U_j - 2}{d_j}, \\ U_{j-1} &= r_j U_j - \frac{2(c_j + d_j)}{b_{j-1}d_j} \quad (2 \leq j \leq K), \\ 0 &= 1 - \frac{2}{d_1} + \left(\frac{a_1}{d_1} + 1 - a_1 \right) U_1 + \sum_{m=2}^K (\beta_m U_m + \gamma_m). \end{aligned}$$

The first equation is obtained from the L_j^- and R_j^+ Bellman equations. The second is obtained from the L_j^+ and R_{j-1}^- Bellman equations. The third is the telescoping identity

$$g(R_0^-) - g(L_{K+1}^+) = V_1 + (g(R_1^+) - g(L_2^+)) + \sum_{m=2}^K (g(L_m^+) - g(L_{m+1}^+)).$$

From the definition of $\mathcal{B}_j$, we have

$$\mathcal{B}_{j-1} = r_j \mathcal{B}_j - \frac{2(c_j + d_j)}{b_{j-1}d_j},$$

so $U_j = P_j \Delta g_K^a + \mathcal{B}_j$ satisfies the second gap equation as $P_{j-1} = r_j P_j$. Substituting $U_j = P_j \Delta g_K^a + \mathcal{B}_j$ into the boundary equation gives exactly the closed form formula from earlier for Δg_K^a . The formula for Δg_j^c can be verified similarly. □

Lemma 13. *The closed forms for μ in B.2.2 solve the target branch equation system.*

Proof. Define

$$M_j := \frac{A_{j-1}C_{j-1}}{B_{j-1}D_{j-1}}, \quad N_j := \frac{A_{j-1}C_j}{B_{j-1}D_j}, \quad \lambda := \frac{a_1c_1}{d_1} P_1.$$

Then $p_{\text{succ}} = \lambda/Z_q$, $N_j = (c_j/d_j)M_j$, $b_j M_{j+1} = a_j N_j$, and $a_j N_j P_j = b_j M_{j+1} P_j = \lambda$. Define

$$\sigma_j := S_j + \frac{a_j c_j}{d_j} P_j, \quad \rho_j := S_{j+1} + P_j.$$

Recall that the closed forms from the above section are $\mu(L_j^+) = M_j \sigma_j / Z_q$, $\mu(R_j^+) = N_j \rho_j / Z_q$, $\mu(L_j^-) = N_j \rho_j / Z_q - \lambda / Z_q$, and $\mu(R_j^-) = \mu(L_{j+1}^+) - \lambda / Z_q$, which we aim to verify below.

First, $\mu(L_1^+) = 1$ because

$$\sigma_1 = S_1 + \frac{a_1 c_1}{d_1} P_1 = \left(1 - a_1 + \frac{a_1}{d_1} \right) P_1 + S_2 = Z_q.$$

Alternatively, we note that the L_1^+ state can only be entered once before it reaches an absorbing state (i.e. the target or back to the fork). Next, using $\sigma_j - \rho_j = \frac{a_j(1-d_j)}{d_j} P_j$,

$$\begin{aligned} c_j \mu(L_j^+) + (1-d_j) \mu(L_j^-) &= \frac{N_j}{Z_q} (d_j \sigma_j + (1-d_j) \rho_j) - \frac{(1-d_j) \lambda}{Z_q} \\ &= \frac{N_j \rho_j}{Z_q} = \mu(R_j^+). \end{aligned}$$

For $j < K$, using $\rho_j - \sigma_{j+1} = (1 - b_j)P_j$,

$$\begin{aligned} (1 - a_j)\mu(R_j^+) + b_j\mu(R_j^-) &= \frac{N_j\rho_j}{Z_q} - \frac{a_jN_j(\rho_j - \sigma_{j+1})}{Z_q} - \frac{b_j\lambda}{Z_q} \\ &= \frac{N_j\rho_j - \lambda}{Z_q} = \mu(L_j^-), \end{aligned}$$

and the same identity for $j = K$ reduces to $(1 - a_K)\mu(R_K^+) = \mu(L_K^-)$, since $a_K N_K P_K = \lambda$ and $\rho_K = P_K = 1$. The equation for L_{j+1}^+ follows similarly:

$$\begin{aligned} a_j\mu(R_j^+) + (1 - b_j)\mu(R_j^-) &= \frac{M_{j+1}\sigma_{j+1}}{Z_q} + \frac{b_jM_{j+1}(\rho_j - \sigma_{j+1})}{Z_q} - \frac{(1 - b_j)\lambda}{Z_q} \\ &= \frac{M_{j+1}\sigma_{j+1}}{Z_q} = \mu(L_{j+1}^+). \end{aligned}$$

Finally, for $j < K$, using $\rho_{j+1} - \sigma_{j+1} = -\frac{a_{j+1}(1-d_{j+1})}{d_{j+1}}P_{j+1}$,

$$\begin{aligned} (1 - c_{j+1})\mu(L_{j+1}^+) + d_{j+1}\mu(L_{j+1}^-) &= \frac{M_{j+1}}{Z_q}((1 - c_{j+1})\sigma_{j+1} + c_{j+1}\rho_{j+1}) - \frac{d_{j+1}\lambda}{Z_q} \\ &= \frac{M_{j+1}\sigma_{j+1} - \lambda}{Z_q} = \mu(R_{j+1}^-). \end{aligned}$$

For $j = K$, the convention for the leaf state yields $\mu(R_K^-) = \mu(L_{K+1}^+) - p_{\text{succ}} = 0$. □

Lemma 14. *The closed forms for $G_j^{(p)}$ for $p \in \{a, b, c, d\}$ in B.2.3 are exactly as stated there.*

Proof. This follows from direct substitution of relevant quantities. □

B.2.5. VALUES AT POST-TRAINING INITIALIZATION

We calculate these values at initialization of post-training, in which $a_j = d_j = \frac{1}{L+1}$ and $b_j = c_j = \frac{L}{L+1}$ for all j . Let us denote $D := 1 + K + \frac{K}{L}$. We will not explicitly show the calculations below, with the understanding that they can be obtained from the closed form expressions from the previous section evaluated at these a_j, b_j, c_j, d_j .

First note that at initialization ($t = 0$), it holds that $p_{\text{succ}} = 1/D$. Now consider the values below at initialization. On the target branch i , we have for $1 \leq j \leq K$:

1. $d_x(L_{i,j}^+) = (K - j + 2) + \frac{K-j+1}{L}$,
2. $d_x(R_{i,j}^-) = (K - j) + \frac{K-j}{L}$,
3. $d_x(R_{i,j}^+) = \left[(K - j + 1) + \frac{K-j+1}{L}\right] \cdot L$,
4. $d_x(L_{i,j}^-) = \left[(K - j + 1) + \frac{K-j}{L}\right] \cdot L = d_x(R_{i,j}^+) - 1$.

Moreover,

1. On a non-target branch, it holds that $d_x(L_j^+) = d_x(R_j^-) = D$ and $d_x(R_j^+) = d_x(L_j^-) = D \cdot L$.
2. The hitting time from a fork state is $H_0 := H_f(0) = (2W - 1)D(1 + K(L + 1))$.

On a non-target branch, it holds at initialization ($t = 0$) that $\tilde{\mu}(s) = 1$ for non-multiedge states and $\tilde{\mu}(s) = L$ for aggregate multiedge states. That is, $\tilde{\mu}(L_j^+) = \tilde{\mu}(R_j^-) = 1$ and $\tilde{\mu}(L_j^-) = \tilde{\mu}(R_j^+) = L$.

For all $1 \leq j \leq K$, it holds that at initialization ($t = 0$):

$$\begin{aligned}\tilde{g}(L_j^+) &= \tilde{g}(L_j^-) = j + (L+1)j(2K+1-j) + \frac{j-1}{L}((L+1)(2K+1-j)+1), \\ \tilde{g}(R_j^+) &= \tilde{g}(R_j^-) = j + (L+1)j(2K+1-j) + \frac{j}{L}((L+1)(2K-j)+1).\end{aligned}$$

Hence, we obtain:

$$\begin{aligned}\Delta_j(0) &= \tilde{g}(L_{j+1}^+) - \tilde{g}(L_j^-) = \frac{2(L+1)}{L}((L+1)(K-j)+1), \\ E_j(0) &= \tilde{g}(R_j^+) - \tilde{g}(R_{j-1}^-) = \frac{2(L+1)}{L}((L+1)(K+1-j)).\end{aligned}$$

For all $1 \leq j \leq K$, it holds that at initialization ($t = 0$):

$$\begin{aligned}\mu(R_j^+) &= \left[\frac{1}{D} \left((K+1-j) + \frac{K+1-j}{L} \right) \right] \cdot L, \\ \mu(L_j^-) &= \left[\frac{1}{D} \left((K+1-j) + \frac{K-j}{L} \right) \right] \cdot L, \\ \mu(L_j^+) &= \frac{1}{D} \left((K+2-j) + \frac{K+1-j}{L} \right), \\ \mu(R_j^-) &= \frac{1}{D} \left((K-j) + \frac{K-j}{L} \right).\end{aligned}$$

For all $1 \leq j \leq K$, it holds that at initialization ($t = 0$):

$$\begin{aligned}g(L_j^+) &= g(L_j^-) = (L+1)(K+1-j) \left(j + \frac{j-1}{L} \right), \\ g(R_j^+) &= g(R_j^-) = (L+1) \left(j(K+1-j) + \frac{j(K-j)}{L} \right).\end{aligned}$$

Hence, we have that:

$$\begin{aligned}\Delta g_j^{(a)} &= g(L_j^-) - g(L_{j+1}^+) = -(L+1) \left((K-2j) + \frac{K-2j+1}{L} \right), \\ \Delta g_j^{(c)} &= g(R_{j-1}^-) - g(R_j^+) = -(L+1) \left((K+2-2j) + \frac{K-2j+1}{L} \right).\end{aligned}$$

For all $1 \leq j \leq K$, it holds that:

$$q(L_j^+)(0) = q(L_j^-)(0) = \frac{K-j+1 + \frac{K-j+1}{L}}{D}, \quad q(R_j^-)(0) = q(R_j^+)(0) = \frac{K-j+1 + \frac{K-j}{L}}{D}.$$

Hence, we obtain:

$$\begin{aligned}\delta_j(0) &= q(L_j^-)(0) - q(L_{j+1}^+)(0) = \frac{1}{D} \cdot \frac{L+1}{L}, \\ \epsilon_j(0) &= q(R_{j-1}^-)(0) - q(R_j^+)(0) = \frac{1}{D} \cdot \frac{L+1}{L}.\end{aligned}$$

Given these quantities, we can now evaluate $G_j^{(p)}$ for $p \in \{a, b, c, d\}$, allowing us to show Lemma 2.

Proof of Lemma 2. We first give the values of the rescaled $G_j^{(p)}$ at initialization. For all depths $1 \leq j \leq K$, it holds at initialization ($t = 0$) that:

$$\begin{aligned} G_j^{(a)}(0) &= \frac{2(L+1)}{LD} ((L+1)^2 j(K-j) + j(W(L^2-1) + 2(L+1)) + (W-1)((L+1)K+1)), \\ G_j^{(b)}(0) &= \frac{2(L+1)}{L^2 D} ((L+1)^2 j^2 - (L+1)(K(L+1) + (L-1)(W-1))j + L(W-1)(KL+K+1)), \\ G_j^{(c)}(0) &= \frac{2(L+1)}{L^2 D} (W(KL+K+L) + (L+1)^2(j-1)(K-j) + (L+1)(j-1)(LW+L-W+1)), \\ G_j^{(d)}(0) &= \frac{2(L+1)}{LD} ((L+1)^2 j^2 - (L+1)((K+1)(L+1) + W(L-1))j + LW(KL+K+L)). \end{aligned}$$

It is easy now to check that $G_j^{(a)}$ and $G_j^{(c)}$ are positive at initialization, whereas the j in the middle depths can possibly cause $G_j^{(b)}$ and $G_j^{(d)}$ to be negative. $\square$

B.3. RLVR Dynamics

We first give a complete proof in the case $L = 1$. In this case there are no multiedges, and a branch is simply a path of length $2K + 1$ from the fork to the leaf. Moreover, it is useful to write $N := 2K$ for this section. For ease of exposition, we will redefine notations (possibly already defined from B.2) to be adapted for use specifically in the $L = 1$ case.

Fix one branch and denote its vertices in order as

$$z_0 = f, \quad z_1, \quad z_2, \dots, \quad z_N, \quad z_{N+1} = t.$$

For $1 \leq j \leq N$, define the forward and backward edge-states

$$F_j := z_{j-1} \rightarrow z_j, \quad B_j := z_{j+1} \rightarrow z_j.$$

We also use the boundary states

$$B_0 := z_1 \rightarrow z_0, \quad F_{N+1} := z_N \rightarrow z_{N+1}.$$

At a forward state F_j , let a_j denote the probability of continuing forward to F_{j+1} . Thus the probability of reversing to B_{j-1} is $1 - a_j$. At a backward state B_j , let b_j denote the probability of continuing backward to B_{j-1} . Thus the probability of reversing to F_{j+1} is $1 - b_j$. Define the two logit gaps

$$D_j^a := \Theta_{F_j, F_{j+1}} - \Theta_{F_j, B_{j-1}}, \quad D_j^b := \Theta_{B_j, B_{j-1}} - \Theta_{B_j, F_{j+1}}.$$

Since there are only two valid next states, the desired probabilities satisfy

$$a_j = \sigma(D_j^a), \quad b_j = \sigma(D_j^b), \quad \text{where } \sigma(u) := \frac{e^u}{1 + e^u}.$$

Let us similarly define the quantities G_j^a and G_j^b to be the $L = 1$ analogues of $G_j^{(a)}$ and $G_j^{(b)}$ from Proposition 2. Then,

$$\frac{dD_j^a}{dt} = \text{sgn}(G_j^a), \quad \frac{dD_j^b}{dt} = \text{sgn}(G_j^b).$$

Target and off-target quantities. For $1 \leq j \leq N$, define

$$\delta_j := q(B_{j-1}) - q(F_{j+1}), \quad U_j := g(B_{j-1}) - g(F_{j+1}),$$

and

$$V_j := \tilde{g}(F_{j+1}) - \tilde{g}(B_{j-1}).$$

Observe that these quantities are the same as we defined from B.1, only that we have adapted it for $L = 1$. In addition, the full hitting-time gap is

$$A_j := h_x(B_{j-1}) - h_x(F_{j+1}) = U_j + \delta_j H. \quad (3)$$

As before, we have that the inequalities $\delta_j > 0$ and $V_j > 0$ follow because every path from F_{j+1} to the fork must pass through B_{j-1} , and every path from B_{j-1} to the target must pass through F_{j+1} .

Furthermore, let

$$\mu_j^F := \mathbb{E} \left[\sum_{t < \tau_{\{f, x\}}} \mathbf{1}\{s_t = F_j\} | s_0 = F_1 \right], \quad \mu_j^B := \mathbb{E} \left[\sum_{t < \tau_{\{f, x\}}} \mathbf{1}\{s_t = B_j\} | s_0 = F_1 \right],$$

be target branch visit counts before absorption at $\{f, x\}$. Define $\tilde{\mu}_j^F, \tilde{\mu}_j^B$ analogously on a non-target branch before hitting f . Suppressing the common positive factor $1/(Wp)$ in Proposition 2, we have

$$G_j^a = \mu_j^F A_j - (W - 1) \tilde{\mu}_j^F V_j, \quad (4)$$

$$G_j^b = -\mu_j^B A_j + (W - 1) \tilde{\mu}_j^B V_j. \quad (5)$$

Closed forms. In this section, we adapt the closed forms from B.1 to the $L = 1$ case. For $1 \leq j \leq N$, define $S_j := \prod_{m=1}^{j-1} \frac{a_m}{b_m}$. Then the off-target visit counts are

$$\tilde{\mu}_j^F = S_j, \quad \tilde{\mu}_j^B = S_{j+1}, \quad (6)$$

which follows from conservation of flux across the cut between z_j and z_{j+1} on a non-target branch. That is, every forward crossing of the cut must be matched by a backward crossing, so

$$a_j \tilde{\mu}_j^F + (1 - b_j) \tilde{\mu}_j^B = \tilde{\mu}_j^B,$$

which is equivalent to $b_j \tilde{\mu}_j^B = a_j \tilde{\mu}_j^F$.

Next, we define $R_j := \prod_{m=1}^{j-1} \frac{b_m}{a_{m+1}}$ for $1 \leq j \leq N$, and $Z_q := 1 + \sum_{r=2}^N (1 - a_r) R_r$. Then observe that

$$\delta_j = \frac{R_j}{Z_q}, \quad p = a_1 \delta_1 = \frac{a_1}{Z_q}. \quad (7)$$

To verify this, subtract the Bellman equations for $q(B_j)$ and $q(F_{j+1})$ to obtain

$$b_j \delta_j = a_{j+1} \delta_{j+1} \quad (1 \leq j < N),$$

so $\delta_j = R_j \delta_1$. By a telescoping decomposition, we have:

$$\begin{aligned} 1 &= q(B_0) - q(F_{N+1}) \\ &= q(B_0) - q(F_1) + \sum_{r=1}^N (q(F_r) - q(F_{r+1})) \\ &= a_1 \delta_1 + \sum_{r=1}^N (1 - a_r) \delta_r = \delta_1 + \sum_{r=2}^N (1 - a_r) \delta_r, \end{aligned}$$

which implies $\delta_1 Z_q = 1$.

For target branch visit counts, define $M_1 := 1$ and recursively

$$M_{j+1} := \frac{a_j M_j - (1 - b_j) p_{\text{succ}}}{b_j} \quad (1 \leq j \leq N). \quad (8)$$

Then

$$\mu_j^F = M_j, \quad \mu_j^B = M_{j+1} - p_{\text{succ}}. \quad (9)$$

The identity $\mu_j^B = M_{j+1} - p_{\text{succ}}$ says that, across the cut at depth j , a successful target attempt has one more forward crossing than backward crossing. Substituting this into the flux equation

$$M_{j+1} = a_j M_j + (1 - b_j) \mu_j^B,$$

gives (8).

The off-target gaps satisfy

$$V_N = \frac{2}{b_N}, \quad (10)$$

and, for $1 \leq j < N$,

$$V_j = \frac{2 + a_{j+1}V_{j+1}}{b_j}. \quad (11)$$

Indeed, subtracting the off-target Bellman equations gives $b_j V_j = 2 + a_{j+1} V_{j+1}$. At the leaf, $\tilde{g}(F_{N+1}) = 1 + \tilde{g}(B_N)$ and the Bellman equation for B_N gives $V_N = 2/b_N$.

For the target time gaps, define $P_1 := 1$, $C_1 := 0$, and recursively

$$P_{j+1} := \frac{b_j}{a_{j+1}} P_j, \quad C_{j+1} := \frac{2 + b_j C_j}{a_{j+1}}.$$

Then

$$U_j = P_j U_1 + C_j, \quad (12)$$

where

$$U_1 = -\frac{N-1 + \sum_{r=2}^N (1-a_r)C_r}{1 + \sum_{r=2}^N (1-a_r)P_r}. \quad (13)$$

The recurrence $a_{j+1}U_{j+1} = 2 + b_j U_j$ is obtained by subtracting the target branch Bellman equations. The scalar U_1 is determined from the telescoping identity

$$0 = g(B_0) - g(F_{N+1}) = N-1 + U_1 + \sum_{r=2}^N (1-a_r)U_r,$$

which gives (13) after substituting (12). Finally,

$$H = \frac{W+1 - a_1 U_1 + (W-1)(1 + a_1 V_1)}{p_{\text{succ}}}. \quad (14)$$

Having specialized all of the quantities to the $L = 1$ case, we now proceed with the analysis of the training dynamics, starting with Phase I.

Phase I. For $1 \leq j < N$, define the following quantity derived from the two terms in G_j^b :

$$\Gamma_j(t) := \frac{(W-1)\tilde{\mu}_j^B(t)V_j(t)}{\mu_j^B(t)A_j(t)}. \quad (15)$$

In particular for $j = N$, we have $\mu_N^B = 0$, and hence $G_N^b = (W-1)\tilde{\mu}_N^B V_N > 0$ whenever the state is reachable. As such, we set $\Gamma_N := +\infty$. It is easy to see the following:

$$G_j^b(t) > 0 \iff \Gamma_j(t) > 1, \quad G_j^b(t) < 0 \iff \Gamma_j(t) < 1.$$

At initialization, for $1 \leq j < N$,

$$\Gamma_j(0) = \frac{(W-1)(N+1)(N-j+1)}{(N-j)(j+(W-1)(N+1))}. \quad (16)$$

We observe that indeed for the middle depths, this can possibly be smaller than 1. In general, both the numerator and denominator will be positive, as these are the contributing terms to the two directions in G_j^b . We now proceed with the main guarantee of Phase I, which is the most technical part of our analysis, yet the shortest phase in empirical simulations.

Lemma 15. *When $L = 1$, the following hold for every depth $1 \leq j < N$:*

(i) (Forward state logit gaps are always increasing): $G_j^a(t) > 0$ for all $t \geq 0$;

(ii) (Negative G_j^b states always make progress towards becoming positive): whenever $\Gamma_j(t) \leq 1$,

$$\frac{d^+}{dt} \log \Gamma_j(t) \geq 2;$$

(iii) (Boundary condition): if $\Gamma_j(t) = 1$, then $\frac{d^+}{dt} \log \Gamma_j(t) > 0$.

Thus, once G_j^b becomes nonnegative, it never becomes negative again.

Proof. Recall that $b_j \delta_j = a_{j+1} \delta_{j+1}$ and $a_{j+1} U_{j+1} = 2 + b_j U_j$. Then, using the fact that $A_j = U_j + \delta_j H$ yields

$$a_{j+1} A_{j+1} = 2 + b_j A_j.$$

In addition, we have that

$$\begin{aligned} a_j G_j^a + b_j G_j^b &= a_j (\mu_j^F A_j - (W-1) \tilde{\mu}_j^F V_j) + b_j (-\mu_j^B A_j + (W-1) \tilde{\mu}_j^B V_j) \\ &= (a_j \mu_j^F - b_j \mu_j^B) A_j \\ &= p_{\text{succ}} A_j, \end{aligned}$$

where in the second line we used the fact that $b_j \tilde{\mu}_j^B = a_j \tilde{\mu}_j^F$ and in the last line we used $a_j \mu_j^F - b_j \mu_j^B = p$. Note that the latter follows from the expected visitation difference between a forward and backward state of the same edge, which is p for an on-target branch as it is exactly equal to the probability we absorb at the target. Since $p_{\text{succ}}, A_j > 0$, we have that $G_j^a > 0$ whenever $G_j^b \leq 0$.

It remains to handle the case of showing the $G_j^a > 0$ when $G_j^b > 0$. We will use the same approach to show both that G_j^b stays positive after it becomes so, and similarly with the G_j^a that correspond to these G_j^b . Concretely, we consider analyzing the regimes where $\Gamma_j(t) < 1$ and $\Gamma_j(t) = 1$. Consider what happens to $\log \Gamma_j$ over time. We differentiate (15). While $\Gamma_j \leq 1$, we have $G_j^b \leq 0$, and from above $G_m^a > 0$ for every m . Hence $\dot{D}_m^a = 1$ for all m , and every m where $G_m^b < 0$ has $\dot{D}_m^b = -1$. Since $\tilde{\mu}_j^B = S_{j+1}$, we calculate:

$$\begin{aligned} \frac{d^+}{dt} \log \Gamma_j &= \frac{d^+}{dt} (\log S_{j+1} + \log V_j - \log \mu_j^B - \log A_j) \\ &= \frac{\dot{S}_{j+1}}{S_{j+1}} + \frac{\dot{V}_j}{V_j} - \frac{\dot{\mu}_j^B}{\mu_j^B} - \frac{\dot{A}_j}{A_j}. \end{aligned}$$

Upon substituting and clearing denominators, we have the following calculation. First, define

$$P_{r,j} := \prod_{m=r}^{j-1} \frac{b_m}{a_{m+1}} \quad (r \leq j), \quad Q_{j,r} := \prod_{m=j}^{r-1} \frac{a_{m+1}}{b_m} \quad (j \leq r).$$

For the target branch terms $-\frac{\dot{\mu}_j^B}{\mu_j^B} - \frac{\dot{A}_j}{A_j}$, we have that

$$-\mu_j^B \dot{A}_j - A_j \dot{\mu}_j^B + \mu_j^B A_j = \mu_j^B A_j + 2p \sum_{r=1}^j P_{r,j} A_r, \quad (17)$$

where we used the fact that

$$b_r V_r = 2 + a_{r+1} V_{r+1}, \quad a_{r+1} A_{r+1} = 2 + b_r A_r,$$

and

$$b_r \tilde{\mu}_r^B = a_r \tilde{\mu}_r^F, \quad a_r \mu_r^F - b_r \mu_r^B = p_{\text{succ}},$$

hold, as well as their time-derivatives. Similarly, for the off-target terms $\frac{\dot{S}_{j+1}}{S_{j+1}} + \frac{\dot{V}_j}{V_j}$ in the time derivative, we have:

$$\mu_j^B A_j \left(\frac{\dot{S}_{j+1}}{S_{j+1}} + \frac{\dot{V}_j}{V_j} - 1 \right) = \frac{2(W-1)S_{j+1}\mu_j^B A_j}{\tilde{\mu}_j^B V_j} \sum_{r=j}^N Q_{j,r} \frac{V_r^2}{V_{r+1} + V_r}. \quad (18)$$

Adding (17) and (18) yields

$$\begin{aligned} & \mu_j^B A_j \left(\frac{\dot{S}_{j+1}}{S_{j+1}} + \frac{\dot{V}_j}{V_j} - \frac{\dot{\mu}_j^B}{\mu_j^B} - \frac{\dot{A}_j}{A_j} \right) \\ &= 2\mu_j^B A_j + 2p_{\text{succ}} \sum_{r=1}^j \left(\prod_{m=r}^{j-1} \frac{b_m}{a_{m+1}} \right) A_r \\ &+ \frac{2(W-1)S_{j+1}\mu_j^B A_j}{\tilde{\mu}_j^B V_j} \sum_{r=j}^N \left(\prod_{m=j}^{r-1} \frac{a_{m+1}}{b_m} \right) \frac{V_r^2}{V_{r+1} + V_r}. \end{aligned}$$

Since $\mu_j^B, A_j > 0$, we have that this time derivative is positive, and hence Γ_j is increasing for all j , as desired. $\square$

We now use Lemma 15 to complete the proof of Lemma 3.

Proof of Lemma 3. Consider a depth j for which $\Gamma_j(0) \leq 1$, or equivalently $G_j^b < 0$ at initialization. Then, by Lemma 15, as long as $\Gamma_j(t) \leq 1$, it holds that $\Gamma_j(t) \geq \Gamma_j(0)e^{2t}$. Hence, G_j^b becomes nonnegative no later than

$$\tau_j \leq \frac{1}{2} \left\lceil \log \frac{1}{\Gamma_j(0)} \right\rceil_+, \quad [u]_+ := \max\{u, 0\}. \quad (19)$$

Consequently, all backward signs are nonnegative by time

$$T_{\text{meet}}^{(1)} := \frac{1}{2} \max_{1 \leq j \leq N} \left\lceil \log \frac{(N-j)(j+(W-1)(N+1))}{(W-1)(N+1)(N-j+1)} \right\rceil_+, \quad (20)$$

where these values come from the $L = 1$ case of B.2.5. $\square$

Phase II. After Phase I, all the G_j^a, G_j^b are now positive. In this section, we will prove the dynamics of Phase II, which ends up being more straightforward than the first phase.

Proof of Lemma 4. At time $T_{\text{meet}}^{(1)}$ which marks the end of Phase I, it holds that all G_j^b are nonnegative (and of course G_j^a will hold regardless). By the boundary condition of $\Gamma_j = 1$ in Lemma 15, we have that the G_j^b will never flip back to being negative. Thus, for all $s \geq 0$,

$$D_j^a(T_{\text{meet}}^{(1)} + s) = D_j^a(T_{\text{meet}}^{(1)}) + 2s, \quad D_j^b(T_{\text{meet}}^{(1)} + s) = D_j^b(T_{\text{meet}}^{(1)}) + 2s.$$

Let $\kappa \in (0, 1/2)$, and define $\Lambda_\kappa := \log \frac{1-\kappa}{\kappa}$. Then, by a straightforward calculation we obtain that in time $T_{\text{meet}}^{(1)} + \Lambda_\kappa$ after Phase II begins, we will have

$$\min_{1 \leq j \leq N} \min\{a_j(T_\kappa^{(1)}), b_j(T_\kappa^{(1)})\} \geq 1 - \kappa.$$

Here, we used the fact that the logit gaps at time $T_{\text{meet}}^{(1)}$ satisfy

$$D_j^a(T_{\text{meet}}^{(1)}) = 2T_{\text{meet}}^{(1)}, \quad D_j^b(T_{\text{meet}}^{(1)}) \geq -2T_{\text{meet}}^{(1)}.$$

Altogether, this proves Lemma 4 for the Phase II dynamics. $\square$

Proof of Theorem 3. We can retrace the proof of the $L = 1$ case to adapt it for the case of general L in the original diamond graph. The main idea is to unfold each diamond into two half-steps. After aggregating over the L symmetric multiedges, the unfolded chain has the same path structure as the $L = 1$ case, with the difference being that the aggregate log-probabilities have fixed offsets of $\log L$.

More precisely, we fix a branch, and define the ordered vertices

$$z_0 := f, \quad z_{2j-1} := u_{j,l}, \quad z_{2j} := u_{j,r} \quad (1 \leq j \leq K), \quad z_{2K+1} := t.$$

Once again, we set $N := 2K$. For $1 \leq m \leq N$, define

$$F_m := z_{m-1} \rightarrow z_m, \quad B_m := z_{m+1} \rightarrow z_m.$$

Then

$$F_{2j-1} = L_j^+, \quad F_{2j} = R_j^+,$$

and

$$B_{2j-1} = L_j^-, \quad B_{2j} = R_j^-.$$

Following the same analysis as before, except with different initialization probabilities of $a_j = d_j = 1/(L+1)$ and $b_j = c_j = L/(L+1)$, we obtain the same result using an identical argument for Phase I. As for Phase II, we observe that the time for all probabilities to be at least $1 - \kappa$ now incurs an extra time of $\log L$ compared to the $L = 1$ case. All in all, this proves convergence towards our learned backtracking policy, and thus concludes the proof of the training dynamics. $\square$

C. Inference Time Separation

In this section, we will prove the separation in inference time compute needed for the SFT model vs. the RLVR model, as formalized by Theorem 4.

Proof of Theorem 4. As noted in 5.3, the RLVR model requires only $\Theta(WK)$ time to find a path. For the converged SFT model in Theorem 2, we recall the following properties of the backtracking behavior that were described in the main text.

1. Upon entering a branch i , since $a_j = c_j = 1$ for all j , we reach the t_i node in $\Theta(K)$ time.
2. When exiting the branch, starting from the state R_{K+1}^+ (i.e. reached the state with head t_i), let g_i denote the expected time of first entry into R_{K+1-i}^- , and f_i denote the expected time of first entry into L_{K+1-i}^- . Then, with the base case of $g_1 = 1$, we have the following recursion:

$$\begin{aligned} f_i &= \frac{L+1}{L} g_i + (2i-1) \cdot \frac{1}{L} + 1, \\ g_i &= (L+1) f_{i-1} + (2i-2) \cdot L + 1. \end{aligned}$$

Hence, the time needed from entry of a wrong branch to getting back to the fork state is g_{K+1} .

We now justify our recurrence equations. Suppose we are currently at the state R_{K+1-i}^- , and we are trying to reach the next state L_{K+1-i}^- . There is a $L/(L+1)$ chance that we proceed forward, and a $1/(L+1)$ chance that we make a u-turn and end up at the leaf again (which takes $2i-1$ time). By standard geometric mean properties, we must make on expectation $(L+1)/L$ attempts before we can successfully get from R_{K+1-i}^- to L_{K+1-i}^- . This tells us that on the first $\frac{L+1}{L} - 1 = \frac{1}{L}$ attempts, we must pay an extra cost of $2i-1$ to go back to the leaf. A similar analysis holds for the case of L_{K+1-i}^- , where the immediate goal is to reach R_{K-i}^- . Here, we must make on expectation $L+1$ attempts, which means the first $L+1-1 = L$ attempts pay an extra $2i-2$ to get back to the leaf.

We will proceed to write down a closed form for g_{K+1} . First, note that:

$$f_{i-1} = \frac{L+1}{L} g_{i-1} + (2i-1) \cdot \frac{1}{L} + 1.$$

Substituting this into the expression for g_i , we have:

$$\begin{aligned} g_i &= (L+1)f_{i-1} + (2i-2) \cdot L + 1 \\ &= (L+1) \left(\frac{L+1}{L} g_{i-1} + (2i-1) \cdot \frac{1}{L} + 1 \right) + (2i-2) \cdot L + 1 \\ &= \frac{(L+1)^2}{L} g_{i-1} + i \cdot \frac{2(L^2+L+1)}{L} - \frac{L^2+L+3}{L}. \end{aligned}$$

Let $r = \frac{(L+1)^2}{L}$, $\alpha = \frac{2(L^2+L+1)}{L}$, and $\beta = -\frac{L^2+L+3}{L}$. Then, our recurrence becomes $g_i = r g_{i-1} + \alpha i + \beta$. By an induction argument, we obtain that:

$$g_{K+1} = r^K + \alpha \sum_{t=2}^{K+1} t r^{K+1-t} + \beta \sum_{t=2}^{K+1} r^{K+1-t}.$$

The first term is of course at least L^K . For the second term, the $t=2$ term multiplied by α already contributes a term of at least $2L^K$. For the third term, it is simply a geometric series, which can be evaluated to be:

$$\beta \cdot \frac{r^K - 1}{r - 1} \geq -2(r^K - 1),$$

for $L \geq 2$. Combining everything, we obtain that

$$g_{K+1} = r^K + \alpha \sum_{t=2}^{K+1} t r^{K+1-t} + \beta \sum_{t=2}^{K+1} r^{K+1-t} \geq L^K + 2L^K - 2L^K = \Omega(L^K).$$

Similar to the RLVR model, we have that by symmetry, it holds that we will have on expectation of W branch entries before reaching the desired target branch. Therefore, the total inference-time compute needed for the SFT model is $\Omega(WL^K)$, and the desired result follows. $\square$

D. Distilling RLVR Reasoning Traces

In this section, we will prove that fine-tuning our pretrained model on the reasoning traces of the RLVR-tuned model will also be effective in mitigating the inefficiency caused by a model trained only on golden examples. We will once again invoke Lemma 5.

Proof of Theorem 5. Let $\mathcal{Q}_{\text{distill}}$ be the distribution over adjacent transition pairs (s, a) induced by $(x, y) \sim \mathcal{D}$; that is, sample (x, y) and then sample a uniformly random transition (y_{t-1}, y_t) from the trace. Then

$$L_{\text{distill}}(\Theta) = \mathbb{E}_{(s,a) \sim \mathcal{Q}_{\text{distill}}} [-\log \pi_{\Theta}(a | s)],$$

and Lemma 5 applies row-wise. In the remainder of the proof, we define the distillation teacher π^* to be the fully converged RLVR policy from Theorem 3. In particular, this means that for any state s and corresponding next state a ,

$$p_{\mathcal{Q}_{\text{distill}}}(a | s) = \pi^*(a | s).$$

By the support assumption in the theorem statement, each such state s satisfies $d_{\mathcal{Q}_{\text{distill}}}(s) > 0$, so the row corresponding to s is updated by gradient flow independently of other rows. Similar to the analyses of pretraining and SFT in the previous sections, it is the case that for each state s , the action space out of s splits into two symmetry classes: the desired action(s) and the undesired action(s). With the pretrained initialization, logits are equal within each symmetry class, and the same ODE uniqueness argument from before implies that they remain equal within each class over time. Thus each such row reduces to a two-logit system (desired logit $u(t)$, undesired logit $v(t)$), and the total desired probability $p(t)$ (which is one of a_j, b_j, c_j, d_j depending on the state type) takes the form

$$p(t) = \frac{m_1 e^{u(t)}}{m_1 e^{u(t)} + m_0 e^{v(t)}},$$

for some class sizes m_1, m_0 (e.g. $m_1 = 1, m_0 = L$ for R_j^+ ; $m_1 = L, m_0 = 1$ for L_j^+ ; etc.).

Because the teacher conditional is supported entirely on the desired class, Lemma 5 gives (up to time rescaling) the same monotone gap dynamics as in the SFT proof. That is, the logit gap $g(t) := u(t) - v(t)$ satisfies $g'(t) \geq 0$ whenever $p(t) < 1$, and in fact the exponential gap $w(t) := e^{g(t)}$ grows at least linearly in t . Consequently $1 - p(t)$ decays to 0, and for any $\kappa > 0$ there exists finite time $T_s(\kappa)$ such that $p(t) \geq 1 - \kappa$ for all $t \geq T_s(\kappa)$.

Applying this argument to every visited state $R_j^+, L_j^+, R_j^-, L_j^-$ and taking $T(\kappa) := \max_s T_s(\kappa)$ over these finitely many state types yields

$$\min_j \min\{a_j(t), b_j(t), c_j(t), d_j(t)\} \geq 1 - \kappa \quad \text{for all } t \geq T(\kappa).$$

In particular, this means the distilled policy inherits the same $\Theta(WK)$ expected hitting-time bound established in Section C for the RLVR policy, as desired. $\square$

E. Additional Experiments

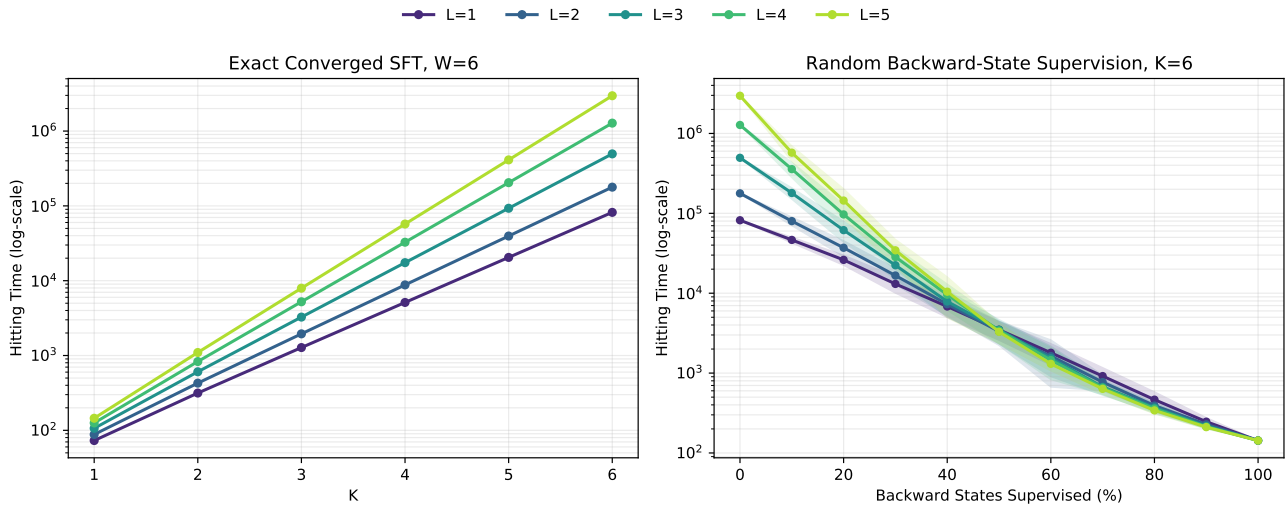


Figure 5. For our SFT experiments, we fix $W = 6$ branches, and plot the hitting times on a log scale. **Left:** For each $1 \leq L \leq 5$, we plot a curve of the (log-) hitting times with respect to the branch length K . **Right:** Here, we additionally fix $K = 6$. For $1 \leq L \leq 5$, we observe the empirical hitting times decrease upon additional supervision of backtracking data. That is, we randomly sample a p fraction of backwards states L_j^- and R_j^- over all $1 \leq j \leq K$, give it supervision through SFT data, and plot the average hitting time under this policy where the supervised backwards states learn to continue backwards. For our experiments, we take the average over 100 random supervisions for each $p = 0, 0.1, \dots, 0.9, 1.0$.

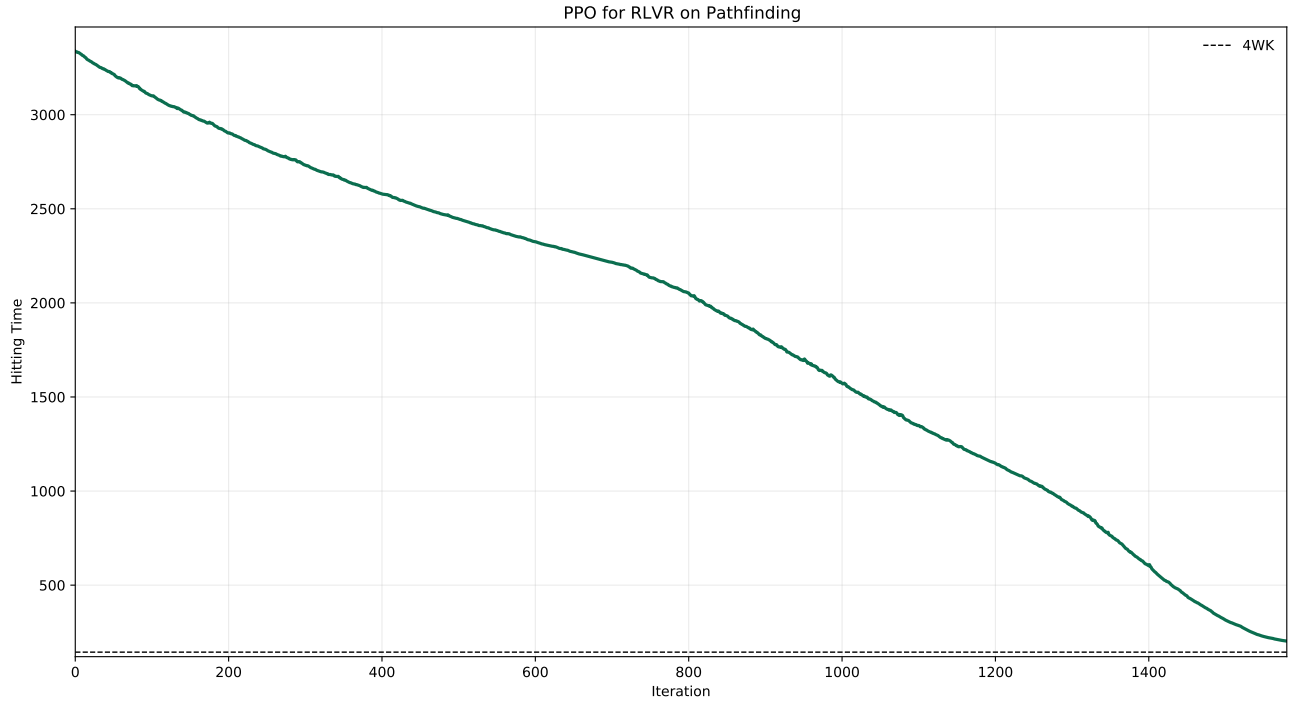


Figure 6. We run PPO on rollout samples for RLVR. Notably, for a given iteration, we use a rollout horizon of two times the current policy’s hitting time of uniform targets, and a very small length penalty of $\beta = 3 \times 10^{-4}$. Our choice ensures we can observe the synergy between the horizon and the length penalty in a more realistic setting. Convergence to the hitting time of $4WK$ (the learned backtracking policy) is shown above.

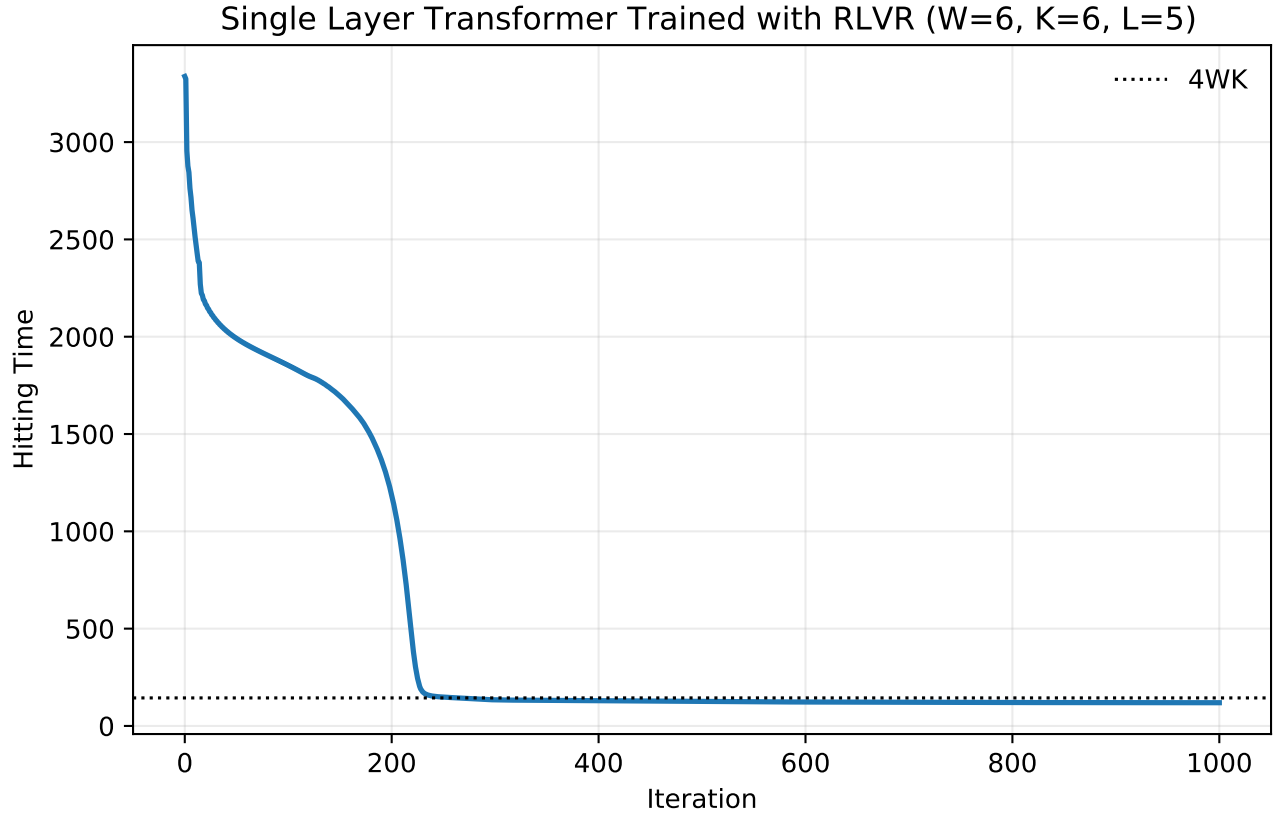


Figure 7. We simulate the training of a single-layer transformer on our pathfinding task. We initialize with the bigram policy as our pretrained policy as per our theory (e.g., by setting the attention matrix equal to identity), and run RLVR via the policy gradient. As predicted by our theoretical analysis, the policy is able to converge to the theoretical optimal of $4WK$ (in fact, it does slightly better than that since a transformer has more expressivity than a bigram model and hence may not have an exact bigram policy).

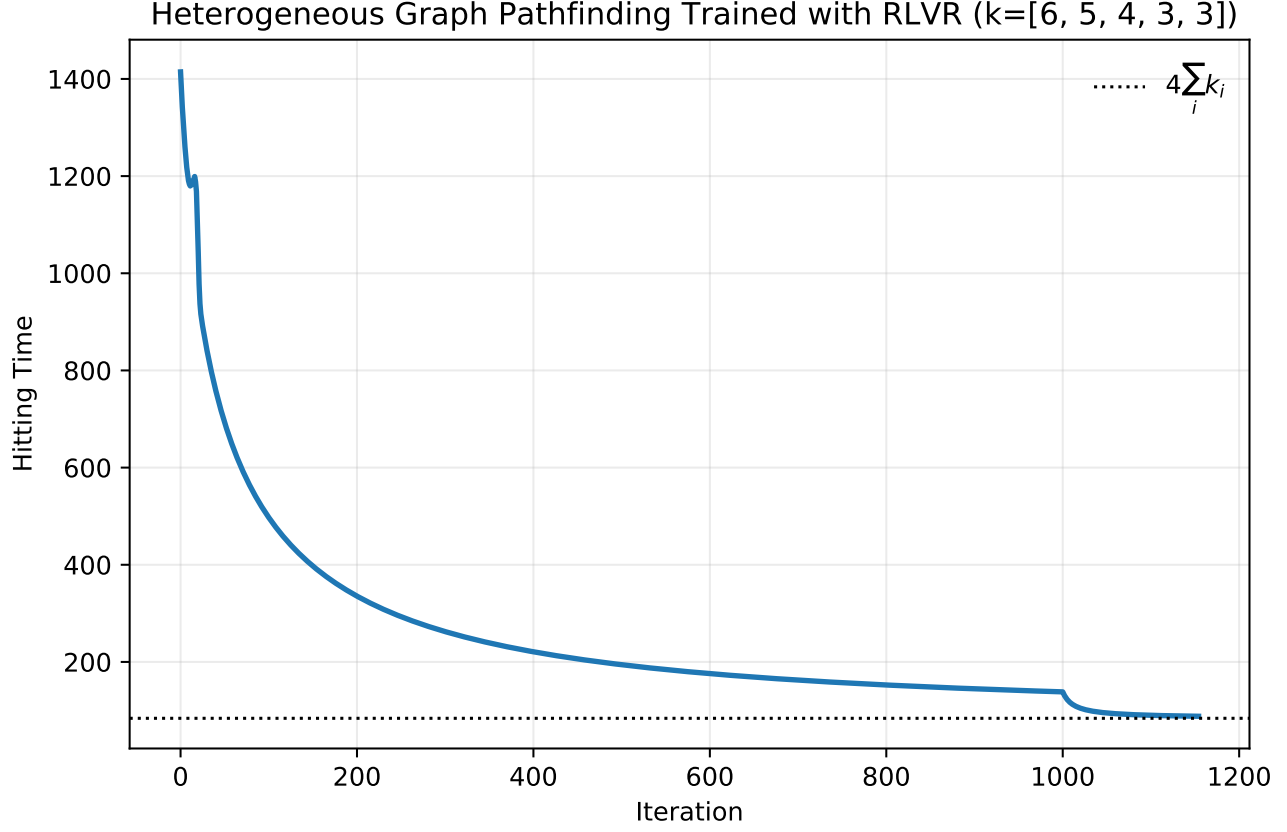


Figure 8. We go beyond the symmetric setting of our paper and analyze the convergence dynamics on a non-symmetric pathfinding task, where different branches are allowed to have different lengths, and different diamonds are allowed to have differing amounts of multiedges. Here, the backtracking target policy will have hitting time $4 \sum_i k_i$, instead of $4WK$. Indeed, RLVR on the policy gradient converges to this policy.